

SIRENBERT: On-Device Detection of Pig-Butchering Scams by Modeling Messages in Conversations

Anonymous Author(s)

Abstract

Pig-butchering is a social-engineering fraud in which a scammer spends weeks building a fabricated relationship over a messaging app before extracting money. Despite the scale of the problem, no detection system exists for pig-butchering: existing detectors target its constituent sub-scams and judge completed conversations, too late to warn a victim, and no labeled dataset follows these conversations message by message. We construct such a dataset, 12,296 conversations spanning real victim transcripts, scammer–scambaiter logs, and benign dialogue, labeled message by message with behavioural triggers by a large language model (LLM) annotation framework restricted to the same message prefix a deployed detector observes. On it we build SIRENBERT, an on-device detector that re-evaluates risk after every incoming message: an encoder maps each message, in context, to a behavioural trigger, and a lightweight recurrent model tracks the conversation’s trajectory through the scam lifecycle. SIRENBERT attains 0.99 held-out conversation SCAM-F1, outperforming keyword, per-message, whole-conversation, and LLM-judge baselines, and warns a median of eleven messages before the first money request, on 76% of scam conversations. We implement SIRENBERT in an Android messaging client; the full pipeline runs on-device, processing each message in under a second.

1 Introduction

Pig-butchering (*sha zhu pan*) is a category of long-con social engineering fraud in which a scammer invests weeks or months cultivating a fabricated relationship with a victim before financially exploiting them. The term originates from the metaphor of “fattening the pig before slaughter”: the scammer builds trust, emotional dependence, and a sense of obligation, then leverages these to extract money [42]. The operation is global and industrial: by tracing on-chain flows from more than 4,000 victims, Griffin and Mei estimate cumulative worldwide pig-butchering losses above \$75B between 2020 and 2024 [26]; in the United States alone, the

FBI reported \$4.57B in investment-fraud losses in 2023, a substantial portion from pig-butchering [18].

These scams take several forms, all sharing a common behavioral lifecycle. Romance fraud, the most prevalent variant, uses a fabricated romantic relationship as the trust-building mechanism [6, 55]. Cryptocurrency mentorship scams involve a scammer posing as a successful trader who guides the victim onto a fraudulent trading platform. Friendship-based exploitation builds trust through platonic companionship before introducing an investment “opportunity” or fabricating a financial emergency. Regardless of the variant, the underlying behavioral trajectory is identical: establish contact, build trust through sustained engagement, deploy deceptive positioning tactics, and ultimately issue a financial demand [17, 51].

The fundamental challenge in detecting these scams is that the trust-building phase is *designed* to be indistinguishable from legitimate conversation. Emotional warmth, personal disclosure, and relationship-building are authentic conversational behaviors; they become scam indicators only in the context of a broader behavioral trajectory that progresses toward exploitation. By the time the financial demand arrives, the victim has been subjected to weeks of manufactured trust, fabricated crises, and emotional manipulation.

No existing system detects pig-butchering as a whole. What exists is detection work on its constituent sub-scams (romance fraud, investment and cryptocurrency fraud), and each of those detection mechanisms fails on pig-butchering in its own way. Keyword and rule-based detectors match financial terms against message text [12]; they fire only once money language appears, after the weeks of grooming that precede it, and they cannot separate an exploitative demand from ordinary money talk, so they warn late on scams and misfire on legitimate conversations. Learned classifiers for the sub-scams judge a *completed* conversation, bag-of-words or term-frequency (TF-IDF) representations of the full transcript [47] or a fine-tuned encoder over the concatenated text; for a fraud whose harm accumulates over weeks of live messaging, a verdict that requires the finished transcript arrives only after the harm is done. General-purpose large language

models (LLMs) also fall short: a measurement of 175,430 user-reported mobile scams finds that off-the-shelf models under zero-shot prompting reach false-positive rates up to 60%, and are least accurate exactly on reply-based romance and wrong-number conversations, the rapport-built scams that constitute pig-butcherings [41]. To our knowledge, no prior system detects pig-butcherings as a whole, and none classifies scam conversations message by message as they arrive.

This work closes that gap with a detector that classifies the conversation as it unfolds, on the user’s own device: after every incoming message it re-computes the verdict from the conversation observed so far, and alongside that verdict it raises an early caution when the scam is still in its setup phase, all without message content ever leaving the phone.

In building and evaluating this system, we answer the following research questions:

- RQ1:** Can reliable message-level supervision for pig-butcherings be produced at scale?
- RQ2:** Can behavioural evidence be extracted reliably from individual messages, and how much conversational context does this require?
- RQ3:** Does reasoning over the behavioural trajectory outperform judging messages or whole conversations?
- RQ4:** How early does the detector warn, and at what false-alarm cost?
- RQ5:** Can the detector be deployed entirely on the user’s device, and at what accuracy and latency cost?

To answer these questions we present SIRENBERT, a streaming, on-device pig-butcherings detection system. SIRENBERT formalises the scam lifecycle as a five-state finite-state machine (FSM) with 14 behavioural triggers, produces per-message silver labels with an LLM annotator that reads each message together with only the conversation up to that point, and trains a two-stage on-device classifier in which a context-aware encoder (Stage 1) emits per-message trigger probabilities and a unidirectional gated recurrent unit (GRU, Stage 2) aggregates them into a conversation-level risk decision.

In answering these questions, this paper makes five contributions:

- We formalise the pig-butcherings lifecycle, described only qualitatively in prior studies, as a five-state machine over per-message behavioural triggers, turning a descriptive account of how these scams unfold into a supervision target a model can be trained on and a state a detector can track message by message.
- We construct, to our knowledge, the first message-level labeled dataset for pig-butcherings detection: 12,296 conversations spanning real victim transcripts, scammer-scambaiter logs, and benign dialogue, in which every

message carries a behavioural label assigned from its preceding context alone. The labels come from a sequential LLM-annotation framework and are validated by cross-checking three architecturally independent model families against one another (κ 0.90–0.95) and against their own whole-conversation judgments ($\kappa \geq 0.98$).

- We design SIRENBERT, a two-stage detector that classifies each incoming message into a trigger and reasons over the running trigger sequence with a recurrent model, so its persistent conversation state is probabilities rather than text. It outperforms keyword, per-message, whole-conversation, and LLM-judge baselines, and warns before the first money request in 76% of scam conversations, a median of 11 messages early, while the terminal SCAM verdict fires on under 1% of benign conversations.
- We show that the LLM modern devices already ship can serve as the per-message classifier while remaining a general-purpose model: prompting it fails outright, but fine-tuning only a small detachable adapter, with the base model untouched, matches the fully fine-tuned encoders.
- We implement the full pipeline inside an Android messaging client and verify it end-to-end by replaying the held-out test set on the device: predictions match the trained model exactly, at 435 ms per message, with raw chat text never leaving the phone.

2 Background and Motivation

This section describes how pig-butcherings operates, defines the threat model we defend against, and establishes why detection must happen while the conversation is still running.

The pig-butcherings lifecycle.

Pig-butcherings is a structured fraud in which scammers cultivate a fabricated relationship over weeks or months before extracting money [42]. Recent victim studies converge on the same progression, opening contact, sustained trust-building, deceptive setup behaviours (credential positioning, platform migration, manufactured crises, investment pitches), and a terminal exploitation phase of one or more financial demands, whether documented from social-media victim reports [1], in-depth victim interviews [46], or automated honey-persona deployments [51]. Across these studies the demand is the end-point, not the bulk, of the scam: the conversation spends most of its length in a scripted setup, credibility claims, platform moves, manufactured urgency, that precedes any mention of money. Figure 1 shows a representative opening: a wrong-number contact drifts through small talk into credential positioning and platform migration, and only later reaches an explicit financial demand. Section 3.1 consolidates these stage descriptions into the five-state lifecycle model our detector operates on.

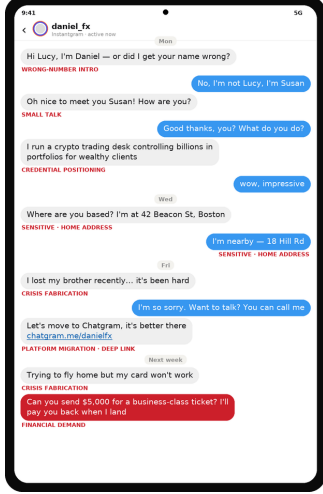


Figure 1: A representative pig-butchering opening on a mobile messaging app, drawn from the lifecycle descriptions in [1, 46, 51].

Threat model.

In this study, the attacker is any party who initiates or sustains a text conversation with a victim with the aim of eventually extracting money from them. The attacker fully controls one side of the conversation: they choose every message, adapt freely to the victim’s replies, sustain the exchange over weeks, and may move it across messaging platforms; we assume the attacker follows the documented pig-butchering playbook but place no constraint on wording, pacing, or persona. The victim is an ordinary messaging user with no prior relationship to the attacker and no reason to suspect the conversation. The stakes of failing this victim are severe: studies document losses of life savings, debt accumulation, emotional trauma, and suicidal ideation once the exploitation completes [1, 46]. Our detector observes the text of the conversation, and only the text: voice calls, images, and videos are out of scope. We additionally evaluate robustness to an attacker who pads the conversation with benign distractor messages (§4).

Why detection must happen during the conversation.

A detector that classifies a conversation only after it has ended, the operating mode of all existing whole-conversation classifiers, arrives too late to protect anyone, for two reasons.

First, harm accumulates while the conversation runs: every additional day invested in the fabricated relationship makes disengagement psychologically harder, the commitment-and-consistency [10] and sunk-cost [31] effects, and once funds are transferred they are rarely recovered. Useful detection must therefore re-evaluate the conversation after every message and raise its caution while the scam is still in its setup phase, when intervention is cheapest; a warning delivered at or after the demand is corrective rather than preventive.

Second, the demand often does not arrive in the conver-

sation being watched: platform migration is a scripted step of the scam, so the exploitation frequently completes in a different app than the one where the grooming happened, in our own real-victim data, 37 of 83 confirmed scam transcripts end before any explicit demand (§3.2). Detection therefore has to act on the setup behaviour it can see, and to follow the conversation across applications when it moves (§5). These two requirements shape the entire design in Section 3.

3 Methodology

This section presents the design of SIRENBERT, which decomposes detection into two stages (Figure 2): a context-aware encoder (Stage 1) reads each incoming message with a window of preceding messages and emits behavioural-trigger probabilities, and a unidirectional GRU (Stage 2) reasons over the accumulated trigger sequence to produce the SCAM/NON_SCAM verdict, together with an auxiliary setup warning, after every message. We first define the lifecycle states and triggers the system predicts (§3.1), then the dataset and annotation that supervise it (§3.2–3.3), and finally the architecture, selection protocol, and deployment (§3.4 onward).

3.1 Lifecycle States and Behavioral Triggers

We model the pig-butchering lifecycle as a five-state FSM with monotonic transitions, consolidated from prior empirical studies of pig-butchering and romance fraud: Acharya and Holz [1], Oak and Shafiq [46], and Spokoyny et al. [51] describe stage decompositions of varying granularity, and Whitty [55] provides the foundational trust-building analysis (per-stage mapping in Appendix A). Our five states mark the two behavioural boundaries common to all four: the transition from ordinary conversation into deliberately deceptive setup, and the transition from setup into explicit financial extraction. Alongside its state, every suspect message receives one of 14 *behavioral triggers*, a label for what the suspect just did, derived from the behavioural patterns these same studies document. Triggers serve two roles: they are the per-message supervision signal Stage 1 learns to predict (Section 3.4), and each trigger admits exactly one state, so the trigger sequence determines the state trajectory. We define each state and its triggers together; the annotation guide’s full exclusion criteria and boundary cases are in Appendix B.

What counts as a scam. In this work a conversation is labeled **SCAM** only if it reaches **EXPLOITATION**, that is, an explicit financial demand is observed in the transcript, and **NON_SCAM** otherwise. The demand is the one exploitation event verifiable from the chat alone; setup behaviour by itself, however suspicious, does not prove that exploitation occurred, and this choice drives the dataset construction in §3.2.

- S_0 **NULL**. no interaction has yet occurred; no trigger maps here.

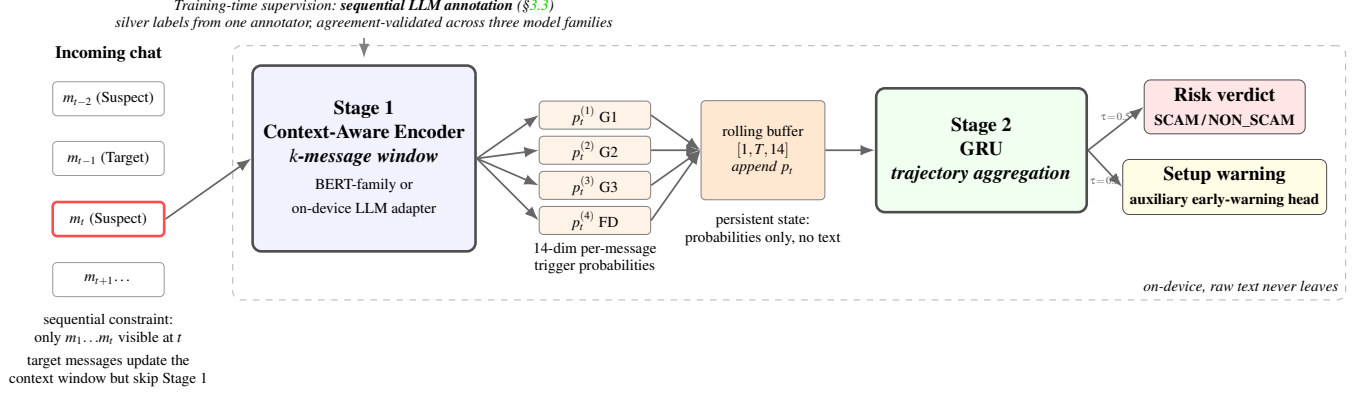


Figure 2: SIRENBERT overview: per-message trigger classification (Stage 1), a rolling trigger-probability buffer as the only persistent state, and sequential aggregation (Stage 2) emitting the verdict and setup warning after every message, entirely on-device.

- **S₁ INITIAL_CONTACT.** the scammer’s first message establishes contact, Oak and Shafiq’s *Lure* [46], Spokoyny et al.’s *Inbound* [51], Acharya and Holz’s *Initial Reach* [1]. Its triggers:

WrongNumber_Intro: contact opened under a wrong-number pretext (“sorry, wrong number, but you seem nice”), the single most common mobile-scam opening in a large-scale Reddit measurement of 2020–2024 [41];

Introduction Opener: a direct self-introduction to a stranger without a prior relationship (“Hi, I’m Anna from Singapore”).

- **S₂ RELATIONSHIP_BUILDING.** the scammer deepens the relationship, the *Bond* stage that Oak and Shafiq observe lasting 3–11 months [46] and that Whitty identifies as the central mechanism of victimisation [55]; on the chat-text side this behaviour is surface-indistinguishable from genuine personal conversation. Its triggers:

Rapport_SelfDisclosure: sharing personal history to invite reciprocal disclosure (“I lost my husband two years ago”) [55];

Affectionate_Language: warmth within the range of ordinary courtship (“good morning dear, thinking of you”);

SmallTalk_Maintenance: the routine conversation that sustains the relationship (“how was your day?”).

- **S₃ PRE_EXPLOITATION.** the conversation has moved beyond rapport into behaviour whose purpose is to prepare exploitation, Oak and Shafiq’s *Bait* [46], Spokoyny et al.’s grooming and cross-platform-request phase [51]. We collapse the setup subphases into one state because they are not reliably ordered in transcripts; the load-bearing boundary is whether setup behaviour has begun at all, and entering this state is the internal warning threshold. Its triggers:

Credential_Positioning: claims of authority, expertise, or wealth deployed to shape trust (“I run a trading team and mentor clients”) [1];

Investment_Pitch: introducing an investment “opportunity” or trading mentorship (“I can teach you how I make 20% a

week”) [42];

Tech_Simulation: walking the victim through a trading platform or app, balances, screenshots, demonstration trades [42];

Platform_Migration: moving the conversation to another application (“let’s talk on Telegram”) [51];

Love_Bombing: disproportionate romantic intensity or premature commitment (“you are my destiny, I have never felt like this”) [55];

Crisis_Fabrication: reporting a sudden personal crisis positioned to elicit concern (“my mother is in the hospital and I cannot reach my bank”) [55];

Emotional_Manipulation: leveraging guilt, obligation, or affection to steer the victim (“if you loved me you would trust me”) [1];

Urgency_Pressure: imposing deadlines or scarcity (“the window closes tonight”) [10].

- **S₄ EXPLOITATION.** an explicit financial demand has been observed, Oak and Shafiq’s *Feed/Squeeze* [46], Spokoyny et al.’s cash-out escalation [51]; funds primarily move through fake investment platforms [42]. This is the terminal state, post-demand stages in prior taxonomies occur outside the chat transcript and add no in-chat signal, and the only state that admits a SCAM verdict. Its trigger:

Financial_Demand: an explicit request for money, gift cards, cryptocurrency, deposits, fees, banking action, or investment (e.g. “put \$500 in and I will trade it for you”). The trigger itself is class-neutral, the same request occurs in benign conversations, and it marks exploitation when it arrives on a trajectory that has passed through setup; learning that distinction is Stage 2’s task (§3.6).

The state transition function is defined as:

$$\delta(s, m) = \max(s, \text{state_of}(m)) \quad (1)$$

where s is the current state and $\text{state_of}(m)$ is the state admitted by the trigger of the current message m . The max operator ensures **monotonicity**: once a conversation reaches a higher

state, it cannot regress, reflecting that once scam tactics have been deployed, the conversation remains suspicious regardless of subsequent benign messages. Transitions may also jump *forward* over intermediate states: δ moves the conversation directly to the highest state the current message warrants, and nothing in the automaton requires passing through the states in between. This is deliberate, because a single message frequently serves several conversational functions at once and the annotator assigns the most advanced behaviour it exhibits: a first message like “Hi, I’m Anna, recently divorced and looking for someone serious” is simultaneously a greeting and personal self-disclosure, so the conversation moves from NULL directly to RELATIONSHIP_BUILDING without ever occupying INITIAL_CONTACT. The transition structure this produces on our dataset is shown in Figure 3 and discussed with the dataset in §3.2.

3.2 Dataset Construction

No public dataset of pig-butcher chat transcripts with conversation-level labels exists, so we assemble proxy datasets that together span real victim dynamics, real scammer behaviour at scale, synthetic pig-butcher trajectories, and benign reference conversations. Because our definition of SCAM requires an observed financial demand (§3.1), we examine every scam-source conversation for one: 665 of the 12,961 collected conversations exhibit setup behaviour but end before any demand, real scam approaches cut short before the defining event, and are therefore neither clean positives nor clean negatives. We exclude them from the dataset and return to them as a held-out stress set (§4.4). The final dataset (Table 1) contains 12,296 conversations: 484 SCAM and 11,812 NON_SCAM. None of these datasets carries per-message behavioural labels; sequential LLM annotation (Section 3.3) supplies that supervision.

LoveFraud02 [40]. 83 real victim-reported romance-scam transcripts ($\approx 31,500$ messages; 30–3,405 messages each) from the LoveFraud.com support community, providing authentic scammer–victim interactions across the full lifecycle. After annotation, 46 reach EXPLOITATION; of the 37 that end before any financial demand (victim disengaged or partial recording), 36 exhibit setup behaviour and fall under the exclusion rule above, and one shows none and is retained as a negative.

Reddit Scambait. 1,166 conversations ($\approx 114,000$ messages; 10–1,773 each) extracted from screenshot posts on r/scambait with Tesseract OCR [50] and manually verified against the source images, following the pipeline of [41]. The scammer side is authentic; the scambaiter side differs from a genuine victim, a distribution shift we evaluate explicitly in Section 4. 438 conversations contain a financial demand; of the 728 that do not, 629 exhibit setup behaviour and are excluded, and 99 are retained as negatives.

Benign datasets. Topical-Chat [23] contributes 8,623

Table 1: Final dataset composition by source.

Source	Convs	Msgs	Avg.	SCAM
LoveFraud	47	33,978	722.9	46
Scambait	537	65,663	122.3	438
Topical-Chat	8,623	188,264	21.8	0
XDailyDialog	3,089	38,911	12.6	0
Total	12,296	326,816	26.6	484

knowledge-grounded conversations whose factual tone is clearly distinct from scam conversations. XDailyDialog [37] contributes 3,089 everyday dialogues (10–40 messages, topic-stratified with a 25% per-topic cap [16]); its warm interpersonal tone is deliberately close to the surface texture of romance scams, so tone alone cannot serve as a shortcut for predicting “not scam” [20, 44].

Train/validation/test split. All experiments split at the conversation level, stratified 80/10/10 over the SCAM/NON_SCAM label. We do not rebalance the classes: a deployed detector sees overwhelmingly benign conversations, so keeping the natural 3.9% scam prevalence in every partition is what makes the measured false-positive rates meaningful. The final dataset splits into 9,836 training, 1,229 validation, and 1,231 test conversations (387/48/49 SCAM).

3.3 Sequential LLM Annotation

Training Stage 1 requires a trigger label for every message, but all source datasets provide only conversation-level labels. Manual labelling of 394,000 messages is infeasible, and rule-based labelling cannot solve the task, because a message’s trigger depends on the conversation that preceded it and on the lifecycle state the conversation has reached. This is precisely the capability profile of instruction-tuned LLMs, whose strengths in understanding intent, context, and multi-step reasoning have motivated their use across scam analysis [52], and recent work shows they can match or exceed crowdworker annotation quality on text-classification tasks [22, 28]. We therefore use Llama 3.3-70B as a silver annotator to convert the heterogeneous raw conversations into a shared message-level format: for each suspect message it produces the trigger, intent, state transition, reasoning, and final conversation classification.

Prompt design and sequential constraint. The annotation prompt contains the FSM state definitions, trigger definitions, boundary rules, and a constrained JSON schema. It explicitly instructs the model to distinguish benign financial language from exploitative demand by using the conversation prefix. The LLM annotates each suspect message seeing only messages m_1 through m_t , never m_{t+1} onward. This sequential prefix constraint matches streaming deployment and prevents future-message leakage in the training labels. It is enforced

by issuing one LLM call per message (batch size = 1).

Does annotation need the future? To test whether the causal constraint costs label quality, we re-annotated the entire held-out test split a second time with the constraint removed: the annotator labels each message while seeing the whole conversation. The two annotations agree almost perfectly on the final conversation verdict (binary Cohen’s $\kappa = 0.98$), and the set of SCAM verdicts is essentially unchanged; the residual disagreement concentrates in borderline pre-exploitation calls, where retrospective access slightly relaxes labels on conversations that end harmlessly. Full-conversation access therefore adds no usable label information, while the sequential constraint keeps the supervision deployment-faithful.

Annotation output. For each suspect message m_t , the LLM produces: a behavioral trigger from the 14-class taxonomy, an intent label (AUTHENTIC or SCAM_INDICATOR), the FSM state before and after this message, and a natural-language reasoning. Output is enforced via guided JSON decoding at the token level, yielding a zero parsing failure rate.

Configuration. Llama 3.3-70B-Instruct [25] serves as the primary annotator, run on four NVIDIA H200 GPUs via vLLM [35] with guided JSON decoding. The context window includes the 15 most recent messages. Maximum output tokens per annotation: 256. Llama 3.3-70B was selected for its 128K-token context window (necessary for conversations up to 3,405 messages), strong instruction-following benchmarks (MMLU-CoT 86.0, IFEval 92.1), full open-source reproducibility, and zero-failure guided decoding via vLLM.

Multi-model agreement. Annotating the full training dataset with several 70B-parameter models would take weeks of GPU time; the held-out test partition is small enough to be annotated by all three. We therefore measure annotation reliability at test-set scale: two additional model families, Qwen2.5-72B-Instruct [56] (dense transformer, different pretraining distribution and tokenizer) and Gemma-4-E4B [21] (the on-device-class family), independently annotate every test conversation under the identical prompt, guided-decoding schema, and sequential prefix constraint as the primary annotator, yielding 15,739 suspect messages per annotator on the dataset. We report pairwise Cohen’s κ [11] on the final conversation verdict, raw per-message trigger agreement, and each annotator’s self-consistency (agreement between its sequential and whole-conversation annotations), interpreting κ on the thresholds of Landis and Koch [36].

Human validation. Human validation uses a stratified subset of 50 conversations, independently annotated by two human annotators using the same trigger taxonomy, state-transition rules, and sequential prefix constraint as the LLM annotators (each annotator labels message t before revealing message $t+1$). We measure Human–Human agreement (does the taxonomy admit consistent human judgements?) and Human–Llama agreement (do the silver labels match human judgement?) using Cohen’s κ [11], following the interpretation

thresholds of Landis and Koch [36].

The annotated dataset. Annotation assigns a trigger to each of the 326,816 messages, turning every conversation into a lifecycle trajectory, and Figure 3 shows what those trajectories look like. The two classes traverse the early states similarly, forward jumps such as NULL $\rightarrow$ RELATIONSHIP_BUILDING are common in both, and separate in whether they ever move beyond relationship building: 87% of NON_SCAM conversations never exhibit any setup behaviour and only 1% ever reach a demand-like utterance, whereas every SCAM conversation reaches EXPLOITATION by definition, most after traversing the setup stage (Appendix E quantifies this divergence).

3.4 Model Architecture

The detection task requires two forms of contextual reasoning that operate at different timescales. At the message level, identifying which behavioural trigger the suspect just deployed depends on *local* context: the same utterance can be benign or exploitative depending on the surrounding messages, and that disambiguation must read the words and the relationship between participants over a small window. At the conversation level, deciding whether the unfolding sequence of triggers constitutes a scam is a *temporal* reasoning problem: the verdict depends on the order in which tactics have appeared and on how setup behaviour accumulates over many messages. A single model that solves both at once would have to fold dense token-level reading and long-range trajectory tracking into one architecture; doing so on commodity mobile hardware is not yet practical. We therefore decompose the classifier into Stage 1 (per-message trigger identification, local context) and Stage 2 (sequence reasoning, temporal context). The decomposition also matches the deployment requirement: raw text is consumed only by the local encoder, while the recurrent component operates on a low-dimensional probability sequence and never sees the underlying text.

The model is trained with two aligned targets. The primary head predicts whether the observed prefix has reached EXPLOITATION, i.e., whether a SCAM verdict is justified from the messages seen so far. The auxiliary head predicts whether the prefix has reached PRE_EXPLOITATION, giving the setup-warning signal used for the early-warning analysis (§2).

3.5 Stage 1: Context-Aware Trigger Classifier

Stage 1 classifies the current message m_t into one of the 14 triggers, reading it together with its k preceding messages; Table 2 (§3.6) walks both stages through an example conversation. Its encoder is BERT-style: every token in the input attends to every other token [14], which is what makes such encoders strong contextual classifiers. This unrestricted attention operates only *inside* the input window, and the window

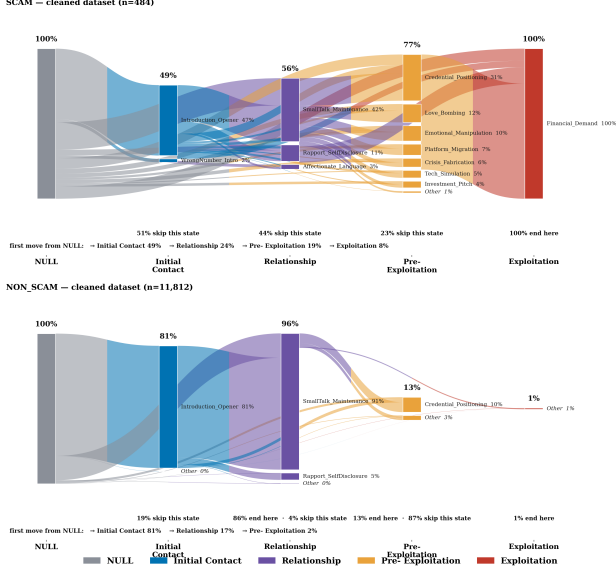


Figure 3: Conversation flow through the lifecycle states on the final dataset, SCAM (top) versus NON_SCAM (bottom); node segments are entry triggers, and all widths are percentages of the class.

contains only the current and preceding messages, so the model can use everything already said in the conversation but nothing that comes after m_t ; at deployment time no future message exists to use, and training matches that condition exactly. The concatenated messages are formatted with BERT’s standard special tokens, the classification token `[CLS]`, whose final-layer representation summarises the whole input, and the separator token `[SEP]`, which marks each message boundary, yielding the encoder input x_t for suspect message m_t :

$$x_t = [\text{CLS}] \ m_{t-k} \ [\text{SEP}] \ \cdots \ [\text{SEP}] \ m_t \ [\text{SEP}] \quad (2)$$

A linear classification head maps the `[CLS]` representation to one of 14 trigger logits. Training labels come from the Llama 3.3-70B silver annotations (Section 3.3). This stage runs entirely on-device: the messages are already displayed on the user’s screen, and the encoder processes them locally without transmitting any text. Conversational context is necessary for the trigger task for two reasons. First, the same sentence means different things in different conversations: “send me \$500” after weeks of love-bombing and credential-positioning is a scam-trajectory financial demand, while the identical sentence from a family member is a mundane request, and only the preceding turns separate them. Second, many messages are too short to interpret in isolation: “ok”, “sure”, or “money” cannot be assigned any trigger on their own. How much context the encoder should read is therefore an open design parameter; we vary the window over $k \in \{0, 1, 2, 3, 4, 5, 10, 15\}$, spanning a current-message-only privacy baseline, the immediate conversational neighbour-

hood ($k \leq 5$), and long-history regimes ($k \in \{10, 15\}$) in which the constructed input can exceed an encoder’s token budget and force truncation. This token budget is what drives the choice of encoder families: we evaluate DistilBERT [49], a compressed BERT variant sized for on-device deployment but capped at 512 tokens, and ModernBERT [54], a long-context encoder with scaled dot-product attention, at both its 512-token and 8k-token limits, so the comparison separates encoder quality from context capacity; the token-length diagnostics in Appendix I confirm that short ambiguous messages benefit most from context and that truncation onset at large k explains the 512-token encoders’ plateau. Both families are fine-tuned end-to-end with the standard BERT hyperparameter configuration [14] (AdamW, linear decay, early stopping on validation macro-F1) without further search, since tuning within this configuration yields marginal gains [45]; exact settings are in Appendix J.

An LLM as the Stage-1 model. We additionally evaluate a generative LLM in the Stage 1 role, for three reasons. First, large-scale pretraining gives LLMs representations and world knowledge that improve predictably with scale [33] and transfer to new tasks from instructions or a handful of examples [5]; on scam classification specifically, this pre-trained knowledge has been observed to generalize to unseen scam types better than task-fine-tuned encoders [52], a property directly relevant to scam behaviours underrepresented in training data. Second, their context windows (128K tokens) exceed every constructed input at any k , removing the truncation regimes that constrain the BERT family. Third, modern phones increasingly ship such a model (e.g. Gemma) as an operating-system service, so a detector could use the model the device already carries instead of shipping its own. Unlike the BERT encoders, however, a generative LLM has no classification head: it produces text. Getting the 14-dimensional trigger distribution out of it therefore requires choosing an extraction mechanism, and we evaluate the standard ones, from zero training to trained adaptation: (i) *prompted scoring*: the frozen model is asked to write a probability per trigger and the written numbers are parsed, the judge-style scoring in which an LLM is prompted to emit risk assessments directly [41, 57]; (ii) *few-shot constrained decoding*: the frozen model receives the annotation taxonomy with worked in-context examples [5] and answers with one of 14 letter tokens, whose next-token probabilities are read off directly, yielding a true probability vector without any training; and (iii) *parameter-efficient fine-tuning*: a LoRA adapter [30] plus a linear head over mean-pooled hidden states, trained on the same labels as the BERT encoders while the base model stays frozen, so the adapter attaches per request and detaches cleanly, preserving the device’s general-purpose model. Because mobile LLM APIs expose generation but not hidden states, we additionally train a generation-only variant of (iii) in which the adapter learns to *emit* the trigger letter itself; this variant is deployable through the standard API at a measured accuracy cost.

The LLM is fine-tuned differently from the BERT families: only a low-rank adapter (rank 64) and the linear head are trained, at learning rate 10^{-4} for three epochs, while the base model stays frozen throughout.

3.6 Stage 2: Sequence Reasoner

Stage 2 converts the per-message trigger evidence into the conversation-level verdict. This conversion is necessary because the defining property of pig-butcherer is the ordered accumulation of behaviours across the conversation: rapport, then credibility claims, then platform moves, then a demand. Every one of those behaviours also occurs in benign conversations, so the scam signal lives in the trajectory rather than in any single message. A financial demand after weeks of love-bombing and investment coaching is exploitation, while an identical request can arise legitimately between people who know each other, repaying a friend, a family member asking for help, and deciding between the two requires a model that has read the whole trigger sequence so far.

At time t , the input is the sequence of trigger-probability vectors $[\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_t]$, with $\mathbf{p}_i \in \mathbb{R}^{14}$; the sequence model receives no other features, so the entire conversation state passed forward is the 14-dimensional probability vector per message, never raw text. The streaming constraint requires a **causal** (left-to-right) sequence model: the verdict at message t may use only messages $1..t$, so bidirectional architectures common in offline classification are out of scope. Table 2 illustrates both stages on a condensed conversation: Stage 1 turns each suspect message into a trigger distribution, and Stage 2’s setup-warning and SCAM probabilities rise as the accumulated trajectory advances through the lifecycle, crossing the warning threshold two messages before the money request.

The task constrains the architecture: Stage 2 must be causal, producing a verdict after every message from the conversation so far, and small enough to run per message on a phone, which makes recurrent models and causally masked attention the viable families; we evaluate a gated recurrent unit (GRU) [9], a long short-term memory network (LSTM) [29], and a causal Transformer encoder at parameter parity. The two recurrent models maintain a fixed-size hidden state over arbitrarily long prefixes at minimal parameter cost (the GRU being the smaller of the two); the causal Transformer attends over the full prefix and is the strongest architecture at long-sequence scale, so including it tests whether recurrence is a limitation at our conversation lengths. Stage 2 emits the primary SCAM head and the auxiliary setup-warning head at every timestep, so a prediction is available after every message rather than only at the end of the conversation.

3.7 Generalization and Source Shift

A single random train/validation/test split measures only *in-distribution* performance: every source contributes some conversations to training, so the test split is well-supported by the training distribution. A real deployment, however, encounters conversations from sources that no training dataset represented, a new dating app, a previously unseen scam playbook, a benign chat platform with different turn-taking conventions. We therefore complement the random split with a grouped leave-one-dataset-out (LODO) protocol that asks: *how does the classifier behave on a source that contributed zero conversations to training?* For each LODO run, one source group is removed from training entirely and used only for testing; the held-out groups are LoveFraud, Reddit Scambait, Topical-Chat, XDailyDialog, and the synthetic pig-butcherer sources, and every system, the compact encoders and both LLM-encoder variants, is retrained under each holdout. LoveFraud and Scambait are held out as complete source groups rather than separating conversations with and without Financial_Demand, because the latter split would test *label-subtype* leakage rather than domain transfer. A second out-of-distribution set comes from the label-hygiene rule itself: the excluded setup-only conversations (§3.2) belong to no split and serve as a stress test for the early-warning channel (§4.4).

3.8 On-Device Deployment

SIRENBERT runs entirely on the user’s device, so raw chat text is never transmitted or retained off-device; we describe the export pipeline and inference loop here and report size, latency, and equivalence measurements in §5.

Export and precision. Both stages are exported from the trained checkpoints to LiteRT (TFLite) with numerical verification at every step, so any on-device divergence is attributable to a specific conversion stage. The encoder is exported in half precision (FP16) at a fixed input length chosen from the data (1,024 tokens covers 99.96% of context windows at the selected k); the FP16 mobile model agrees with the trained checkpoint on 100% of predicted triggers on held-out inputs, so the export chain is argmax-lossless. Stage 2 is exported as a per-timestep unrolled GRU emitting the SCAM and setup-warning logits at every step, which we verify reproduces the dynamic-length reference exactly at every prefix length. Dynamic INT8 quantization was evaluated as a smaller-footprint alternative and rejected: it is the isolated cause of an on-device recall drop (0.804 to 0.686 in bulk replay), reported as an ablation in §5.

On-device inference loop. At runtime the device performs, per incoming message: (i) WordPiece tokenization of the current message together with its k preceding messages (reimplemented on-device, no server tokenizer); (ii) a Stage 1 forward pass producing the 14-dimensional trigger-probability

Table 2: Both stages on a condensed example conversation (illustrative probabilities; row maximum in bold). Trigger columns: WN=WrongNumber_Intro, IO=Introduction_Opener, RS=Rapport_SelfDisclosure, AL=Affectionate_Language, ST=SmallTalk_Maintenance, CP=Credential_Positioning, IP=Investment_Pitch, TS=Tech_Simulation, PM=Platform_Migration, LB=Love_Bombing, CF=Crisis_Fabrication, EM=Emotional_Manipulation, UP=Urgency_Pressure, FD=Financial_Demand. The last two columns are Stage 2’s setup-warning and SCAM probabilities after the message.

<i>t</i>	Suspect message	WN	IO	RS	AL	ST	CP	IP	TS	PM	LB	CF	EM	UP	FD	Warn	SCAM
1	“Hi, I’m Anna from Singapore”	.01	.72	.05	.01	.19	.01	.00	.00	.00	.00	.00	.00	.00	.01	.02	.00
2	“how was your day?”	.00	.02	.12	.03	.81	.01	.00	.00	.00	.00	.00	.00	.00	.01	.03	.00
3	“I run a trading team, mentor clients”	.00	.01	.04	.01	.08	.64	.21	.01	.00	.00	.00	.00	.00	.00	.38	.02
4	“I can teach you, 20% a week”	.00	.00	.01	.00	.02	.14	.77	.03	.00	.00	.00	.00	.02	.01	.81	.08
5	“put \$500 in, I trade it for you”	.00	.00	.00	.00	.00	.01	.07	.02	.00	.00	.00	.00	.02	.88	.97	.93

vector; (iii) an append to a per-conversation rolling buffer of trigger vectors; and (iv) a Stage 2 forward pass over that buffer producing the SCAM and setup-warning probabilities. Target (non-suspect) messages update the context buffer but skip Stage 1, mirroring the training-time information flow. Figure 4 illustrates this loop on a chat prefix: Stage 1 assigns a trigger distribution after each message and Stage 2 consumes the accumulated vectors to update the conversation risk. Crucially, the raw text is not the long-term state passed to the sequence reasoner, the on-device buffer stores only trigger probabilities, so the sequence state never contains raw message text.

Runtime and reference integration. The deployed engine runs the FP16 LiteRT models through the standard mobile interpreter. To validate end-to-end feasibility we integrate the engine into an Android messaging client (Element X) behind a feature flag, classifying live conversations on-device, and additionally port the identical engine to a second, independently maintained client (SchildiChat X) to demonstrate that the integration surface is client-agnostic. Because platform migration is a scripted step of the scam lifecycle, the two clients also implement conversation continuity: on a Platform_Migration trigger the source app emits a deep link carrying only an opaque conversation key, the receiving app seeds its tracking buffer under the same key, and risk state keeps accumulating with no message content crossing the application boundary; a notification-listener service extends coverage to apps without native integration.

Operating points. Selection yields one operating point per encoder family; because the points are not statistically separated in accuracy, we report size, on-device latency, and error trade-offs for each and treat the choice among them as a deployment decision (§5).

4 Results

This section first validates the annotation that supervises the system (RQ1), then evaluates Stage 1 and the selection of its configuration (RQ2), the full system against all baselines (RQ3), the warning lead time (RQ4), and generalization; deployment (RQ5) follows in Section 5. All metrics treat SCAM

as the positive class and are computed at the conversation level: a false positive is a benign conversation flagged as SCAM, and a false negative is a scam conversation the system fails to flag.

4.1 Annotation Reliability

Llama 3.3-70B annotates the full dataset and produces the silver labels used to train Stage 1 and Stage 2. Reliability is measured at test-set scale as described in §3.3, in the same spirit in which multi-model comparison is used to characterise LLM reliability on scam classification [52]: three model families annotate every held-out test conversation independently (1,222 conversations; 15,739 suspect messages per annotator). Table 3 summarises the results.

Every pairwise verdict κ is at or above 0.90, “almost perfect” on the Landis–Koch scale [36]: three architecturally independent model families, including an on-device-class model an order of magnitude smaller than the others, converge on the same conversation-level judgements. Self-consistency is higher still ($\kappa \geq 0.98$), confirming that the sequential constraint does not inject instability relative to whole-conversation annotation. Per-message trigger agreement (82.6–85.2%) is high for a fine-grained 14-class task; the structure of the residual disagreement matters more than its level, and it is benign (Section 4.1). Human validation answers a different question from cross-model reliability and is reported separately below.

Trigger disagreement. Pooled over the three annotator pairs (47,210 pairwise message comparisons), triggers agree on 83.7% of messages, leaving 16.3% in disagreement. Of those disagreements, 39% are between triggers that admit the *same* lifecycle state (e.g. SmallTalk_Maintenance vs Rapport_SelfDisclosure) and therefore change neither the conversation label nor detection timing; only 9.3% involve the decision-critical Financial_Demand trigger; the remainder concentrate at the RELATIONSHIP_BUILDING $\leftrightarrow$ PRE_EXPLOITATION boundary, the same boundary that defines the setup warning (per-pair values in Table 3).

Human validation. Two independent human annotators labelled a stratified 50-conversation subset under the same sequential prefix constraint as the LLM annotators (§3.3); Ta-

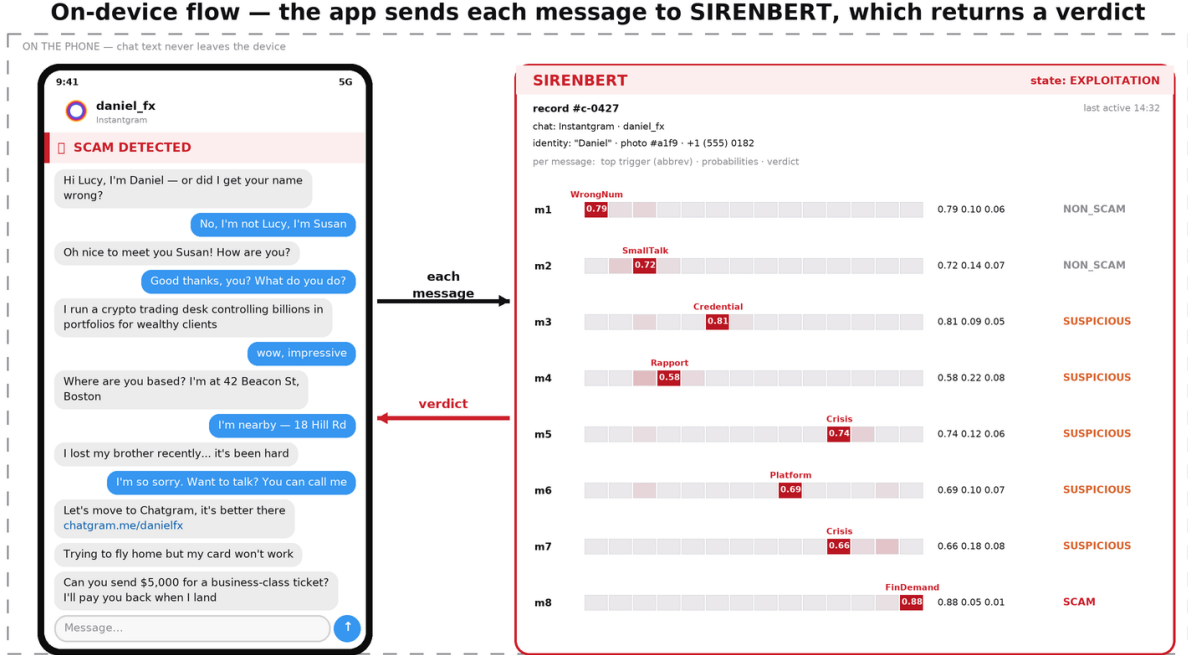


Figure 4: On-device Stage 1-to-Stage 2 progression. The app (left) passes each message to the on-device SIRENBERT record (right), which holds, per message, the top Stage 1 trigger (with abbreviation and probabilities, the only state retained on-device) and the resulting per-message state (no alert / setup warning / SCAM). Raw message text is consumed by Stage 1 but never enters the persistent sequence state.

Table 3: Annotation reliability at test-set scale (1,222 held-out conversations; 15,739 suspect messages per annotator).

Rater pair	Verdict κ	Trigger agreement
Llama $\leftrightarrow$ Qwen	0.95	85.2%
Llama $\leftrightarrow$ Gemma	0.90	82.6%
Gemma $\leftrightarrow$ Qwen	0.90	83.2%
<i>Self-consistency (sequential vs whole-conversation)</i>		
Llama	0.98	90.4%
Gemma	0.99	90.4%
Qwen	0.98	90.3%

Table 4: Human validation on the 50-conversation subset (sequential prefix constraint).

Comparison	Trigger κ	Verdict agreement
Human $\leftrightarrow$ Human	[TBD]	[TBD]
Human consensus $\leftrightarrow$ LLM	[TBD]	[TBD]

access changes verdicts almost nowhere ($\kappa = 0.98$), so restricting the annotator to the deployed detector’s information costs no label quality.

ble 4 reports agreement between the humans (the ceiling any annotator can reach) and between the adjudicated human consensus and the silver labels. The acceptance criterion, fixed before adjudication, is that human–LLM agreement be statistically indistinguishable from human–human agreement; trigger classes violating it are re-prompted.

RQ1 takeaway: causal supervision

Prefix-constrained LLM annotation is reliable enough to train on: three independent model families agree on held-out verdicts at κ 0.90–0.95, each annotator is self-consistent at $\kappa \geq 0.98$, and granting full-conversation

4.2 Financial-Demand Baseline Results

Two baselines motivated the design of SIRENBERT and are reported first: they test whether pig-butcher detection can be solved by simply waiting for a Financial_Demand utterance, a regex detector and a per-message demand classifier, each aggregated by the rule a demand-only detector would use (predict SCAM if any message is flagged). Their failure modes are what the full behavioural-trigger design answers; the comparison baselines for the finished system (LLM judges, whole-conversation fine-tune) appear alongside it in §4.

Regex baseline. A keyword detector flags messages containing financial terms (“send money,” “gift card,” “wire transfer,”

and related patterns). A conversation is classified as SCAM if any message triggers a match. This baseline tests whether scams can be detected by financial vocabulary alone.

Per-message classifier for Financial_Demand. Per-message classification is the standard formulation in adjacent moderation problems such as spam, phishing, and abusive-content detection: a fine-tuned encoder scores each message independently and a policy aggregates the per-message scores [14, 39]. We adapt this formulation to the demand-only detection question. Each training message receives a binary label, contains a financial demand or not, derived from the dataset annotation (§3.3); we fine-tune DistilBERT [49] on these single messages with no surrounding context, matching what a per-message detector sees at deployment. At evaluation, every message of a test conversation is scored independently, and the conversation is classified SCAM if any message is predicted to contain a financial demand. In a preliminary four-model comparison (BERT-base [14], DeBERTa-v3 [27], DistilBERT, FinBERT [3]), all four were within one point of one another at both the message and conversation level, so we carry the smallest (DistilBERT) forward as the representative per-message baseline.

Table 5 reports the demand-only baselines on the held-out test split (48 SCAM, 1,249 primary NON_SCAM conversations), the same split as every later experiment. At evaluation time, each detector is applied to every message and aggregated with the deployment rule a demand-only detector would use: predict SCAM if *any* message is flagged as Financial_Demand. Setup-only conversations are labeled NON_SCAM, so a demand-only baseline that escalates one is counted as a false SCAM prediction.

The learned per-message classifier dominates the regex on precision (0.222 $\rightarrow$ 0.636) at equal-or-better recall, confirming that learned semantic features separate demand-shaped language from incidental financial vocabulary far better than keyword matching. The learned classifier nevertheless remains limited: its held-out message-level FD-F1 is 0.630, demand identification is itself context-dependent, and 24 benign conversations are still falsely escalated because an any-demand rule cannot distinguish a benign request for money from an exploitative one. Detecting the scam therefore requires classifying the surrounding behaviour rather than the demand alone, which is the task the remainder of this section evaluates.

4.3 SIRENBERT Classifier Performance

We now evaluate SIRENBERT, the final two-stage classifier and sequence reasoner, on the active dataset. This split contains 12,961 conversations and 209,164 annotated suspect messages. The conversation-level stratified 80/10/10 split (Table 17, Appendix J) preserves the dataset SCAM prevalence of 3.7% (3.8% validation, 3.7% test).

The experiments proceed in the order the system was de-

Table 5: Demand-only baselines on the held-out test split (48 SCAM, 1,249 NON_SCAM); conversation-level precision, recall, and F1 under the any-demand rule.

Method	Precision	Recall	F1	FP	FN
Regex keywords	0.222	0.813	0.348	137	9
DistilBERT FD-only	0.636	0.875	0.737	24	6

signed: Stage 1 first (how much context does the trigger task need, and which encoder), then the model-selection protocol, then Stage 2 and its ablations.

4.3.1 Stage 1: Context Window and Encoder Results

We sweep every encoder over the full context grid $k \in \{0, 1, 2, 3, 4, 5, 10, 15\}$ under 5-fold nested cross-validation and report the complete sweep together; the questions the sweep answers are whether context helps the trigger task, at what window size the benefit saturates, and what bounds it. Context helps every encoder: on the compact 512-token encoder, trigger macro-F1 climbs from 0.49 at $k=0$ to 0.53 at $k=5$, then plateaus through $k=10$ and drops at $k=15$. The plateau is a token-budget effect, not a property of context itself: as the window grows, an increasing fraction of constructed inputs exceeds the 512-token limit and truncates, which we confirm with per-message diagnostics in Appendix I.

Why context and length matter. Two per-message diagnostics, reported in full in Appendix I, explain why context helps and what limits it. The shortest messages (“ok”, “money”) are uninterpretable in isolation and gain steeply with k , while a long message plus a long context exceeds the 512-token cap, and because the target message sits at the end of the constructed input, truncation removes the very message being classified. Constructed inputs first exceed the cap at scale at $k=15$ ($\approx 8\%$ of inputs), exactly where 512-token accuracy on long messages collapses; this motivates an 8k-token encoder.

4.3.2 Stage 1: Encoder Comparison

The token-length analysis justifies a second encoder and dictates its requirement: it shows our constructed inputs exceed DistilBERT’s 512-token cap at large k (negligibly through $k=5$, $\approx 8\%$ of inputs at $k=15$), and truncation is what drags accuracy down on long inputs. We therefore need an encoder whose token budget covers our longest constructed inputs. We choose ModernBERT-SDPA [54]: it is trained natively for long-context representation (8,192 tokens, covering the 99th percentile of our $k=15$ inputs), and, unlike long-context alternatives that change the attention pattern, it remains a standard BERT-style encoder, so the comparison isolates context capacity rather than confounding it with an architectural family change. We re-run the identical context sweep at two

token budgets (Figure 5): ModernBERT-SDPA capped at 512 tokens isolates encoder quality at the same budget as DistilBERT, and ModernBERT-SDPA at 8k isolates the value of the longer budget itself. For calibration, both reference baselines are far below every learned cell: always predicting the majority trigger yields macro-F1 ≈ 0.05 and uniform-random ≈ 0.07 (the dashed and dotted lines in Figure 5); all encoders clear both by a wide margin at every k .

ModernBERT-SDPA improves over DistilBERT under both conditions the comparison was designed to test. At the same 512-token budget it reaches macro-F1 / accuracy of .55/.85 at $k = 5$, ahead of DistilBERT’s .53/.84, a strict gain at an identical input format. At the 8k-token budget it reaches the best Stage 1 macro-F1 of .57/.86 (at $k = 5$ and $k = 15$) and, unlike the 512-token encoders, keeps gaining on the longest windows rather than saturating.

The LLM encoder. The same LLM spans failure to competitiveness depending on how it is used: prompted scoring fails outright (macro-F1 0.06, the written numbers are fabricated), few-shot constrained decoding remains weak (0.27), the fine-tuned adapter is competitive with the dedicated encoders across the sweep and peaks at $k = 10$, and the generation-only letter variant trades a measured amount of accuracy for deployability through mobile LLM APIs. What separates these outcomes is training on the task; a trained adapter makes the device’s own model a viable Stage 1.

From Stage 1 to Stage 2. All encoder sweeps, both BERT families and the fine-tuned LLM adapter, feed Stage 2, and the operating points are selected by the protocol below (§4.3.3); the configurations that emerge are reported in Table 6, with the $k = 0$ cells retained as no-context privacy baselines.

4.3.3 Model Selection Protocol

We select the operating configuration, the encoder and its context window, by *nested* 5-fold stratified cross-validation over the pooled training and validation conversations (≈ 87 SCAM conversations per held-out fold): the Stage 1 encoder is re-trained per fold so each fold is scored by an encoder that has never seen it, and the test split is never used for selection, being reported once for the selected configuration only [7, 48, 53]. Because encoder fine-tuning varies run to run even at fixed hyperparameters [15, 45], each configuration is repeated over three a-priori seeds (7, 42, 123) and reported as the mean of the 15 fold-seed estimates [34].

Picks. We report picks under two different metrics, because the two stages of the system have different objectives. The *Stage 1 picks*, selected by mean nested-CV per-message trigger macro-F1, averaged over the 5 held-out folds, are: DistilBERT $k = 5$ (macro-F1 0.527 ± 0.013), ModernBERT-SDPA 512 $k = 10$ (0.550 ± 0.009), and ModernBERT-SDPA 8k $k = 15$ (0.552 ± 0.007). The *end-to-end picks*, selected by mean nested-CV conversation SCAM-F1 after Stage 2, averaged over the 15 fold-seed estimates per configuration, are: Distil-

BERT $k = 1$, ModernBERT-SDPA 512 $k = 5$, ModernBERT-SDPA 8k $k = 10$, and the LLM adapter $k = 5$. The deployed system uses the end-to-end picks, because the deployment artifact is the joint Stage 1 + Stage 2 pipeline rather than Stage 1 alone. The LLM encoder enters selection with its context window chosen on its own sweep, since re-training a multi-billion-parameter fine-tune inside every fold of the nested grid would cost months of GPU time; its end-to-end configuration is evaluated identically from that point on.

Significance and robustness. A paired Wilcoxon signed-rank test over the 15 fold-seed estimates, Bonferroni-corrected over three metrics, finds that only five of the 36 adjacent- k drops are statistically significant, four of them coinciding with the 512-token truncation onset, and re-training each pick from scratch under two alternative 80/10/10 splits moves test SCAM-F1 by at most 0.020, within the bootstrap interval of the canonical number [24]. The selected windows therefore reflect token-budget effects rather than sampling noise; the per- k tables, the full significance analysis, and the $k = 0$ no-context baselines are in Appendix J.

4.3.4 Stage 2: Sequence-Reasoning and Seed Stability

Stage 2 turns the per-message evidence into the conversation verdict, so its evaluation asks three things: whether reasoning over the order and accumulation of triggers improves the final decision beyond message-level evidence alone, whether that gain is stable across training seeds, and which causal architecture should provide it.

Since the positive class of the test fold is structurally small, we quantify uncertainty before interpreting the point estimates in Table 6. Standard deviations across the three training seeds stay at or below 0.005, so the operating points are not artifacts of a particular initialisation, whereas the bootstrap confidence intervals remain wide and overlap across the encoder families, and an exact McNemar test on per-conversation predictions finds no significant difference between any two of them ($p \geq 0.12$). We therefore report all configurations rather than ranking them, and treat the choice among them as a deployment trade-off between model size and error profile.

RQ3 takeaway: encoders and context

Conversational context helps the trigger task up to a window of roughly five to ten preceding messages and is bounded by each encoder’s token budget; under nested cross-validation the picked configurations of all encoder families are statistically inseparable, so the operating choice is a deployment-size trade-off rather than an accuracy ranking.

4.3.5 Comparison Against Baselines

We now evaluate the full system against every baseline family on the same held-out split: the demand-only detectors of §4.2,

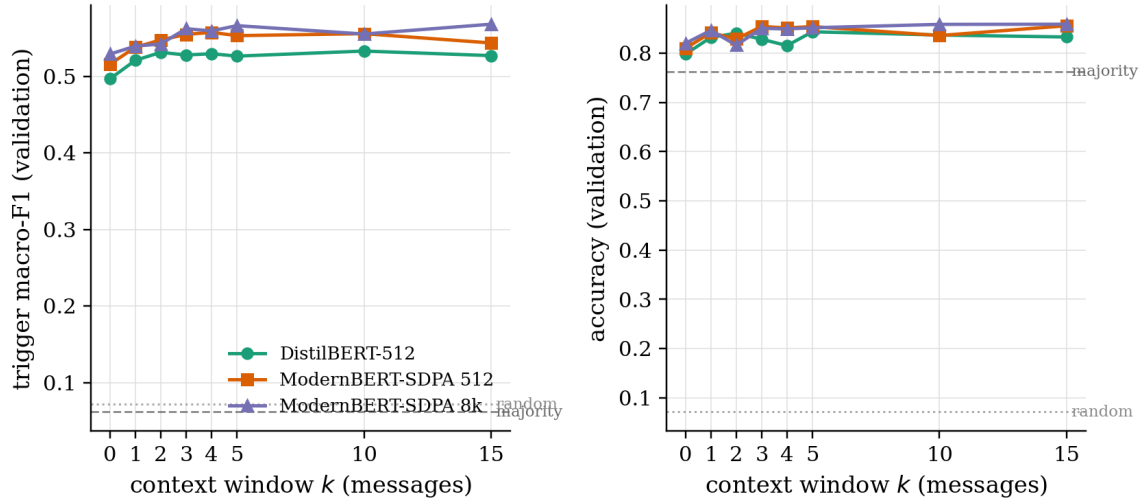


Figure 5: Stage 1 trigger macro-F1 (left) and accuracy (right) across context windows k (5-fold nested-CV mean $\pm$ std); dashed and dotted lines are the majority-class and uniform-random baselines.

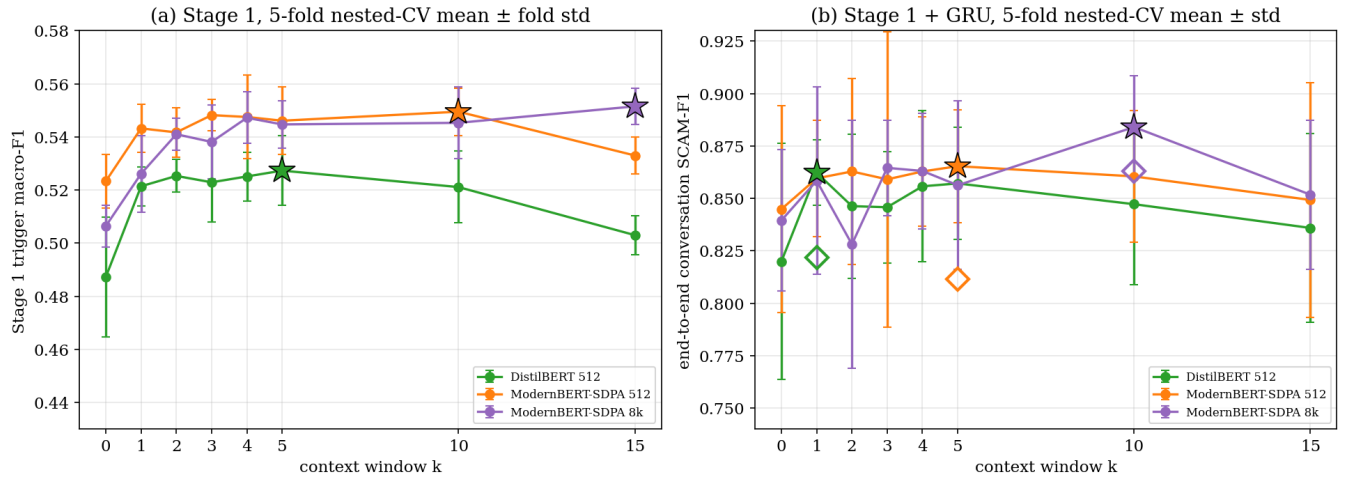


Figure 6: Nested cross-validation selection signals across k : (a) Stage 1 trigger macro-F1 and (b) end-to-end conversation SCAM-F1 (5-fold mean $\pm$ std; stars = picks, diamonds = held-out test at the pick).

Table 6: Stage 2 conversation-level results on the held-out test split (49 SCAM, 1,182 NON_SCAM): mean $\pm$ std over three seeds; bootstrap 95% CI at seed 42 (10,000 resamples).

Encoder + GRU	k	Test SCAM-F1	95% CI	Precision	Recall
DistilBERT 512	1	0.967 $\pm$ 0.004	[0.917, 0.991]	0.92	1.00
ModernBERT-SDPA 512	5	1.000 $\pm$ 0.000	[1.000, 1.000]	1.00	1.00
ModernBERT-SDPA 8k	10	0.993 $\pm$ 0.005	[0.965, 1.000]	1.00	0.98
LLM adapter	5	0.986 $\pm$ 0.005	[0.964, 1.000]	1.00	0.98
LLM adapter (letter)	5	0.983 $\pm$ 0.005	[0.947, 1.000]	0.96	1.00

Stage 1 alone under any-demand aggregation, and the two whole-conversation baselines introduced below.

Whole-conversation fine-tune (conv-BERT). The standard formulation from prior sub-scam work is a single encoder fine-tuned on the concatenated transcript with one conversation-level label [47]; we train this baseline on the identical splits and labels, and it tests whether the trigger taxonomy and the sequence model are needed at all.

Whole-conversation LLM judge. Prompting an instruction-tuned LLM to render a verdict over a complete input, popularised as “LLM-as-a-judge” by Zheng et al. [57], is a natural whole-conversation baseline, and prior work has already applied zero-shot LLM judgement to scams: Lu et al. [41] report false-positive rates up to 60% precisely on the romance and wrong-number dialogues central to our setting, and Topcuoglu et al. [52] find even the strongest of nine benchmarked models reaches only $\sim 65\%$ micro-F1 on single-item scam classification. We therefore include judge baselines under a deliberately favourable protocol: the model receives the *complete* conversation and answers a single binary SCAM / NON_SCAM question, with chat-template formatting and deterministic decoding. We evaluate one server-class judge (Llama 3.3-70B) and one on-device-class judge (Gemma-4-E4B, the model family shipped by mobile inference stacks); the two perform comparably on our dataset and both are reported. Two properties bound what this baseline can show: the judge reads the finished conversation, including any financial demand, so it measures post-hoc classification rather than during-conversation detection; and running it per message would require re-processing the entire conversation on every new message.

Figure 7 ranks every baseline and system configuration on the held-out test split: keyword matching sits far below everything learned, whole-conversation models and Stage 1 alone occupy the middle, and adding the sequence model lifts every encoder family, including the LLM adapter, to the top of the ranking, because the GRU learns when a demand-like trigger belongs to a scam trajectory and when it should not escalate. The conv-BERT fine-tune is competitive on this final-verdict metric but produces a single post-hoc label: it cannot say which message constituted setup behaviour, cannot warn before the demand (§4.3.6), and re-encodes the growing transcript on every message.

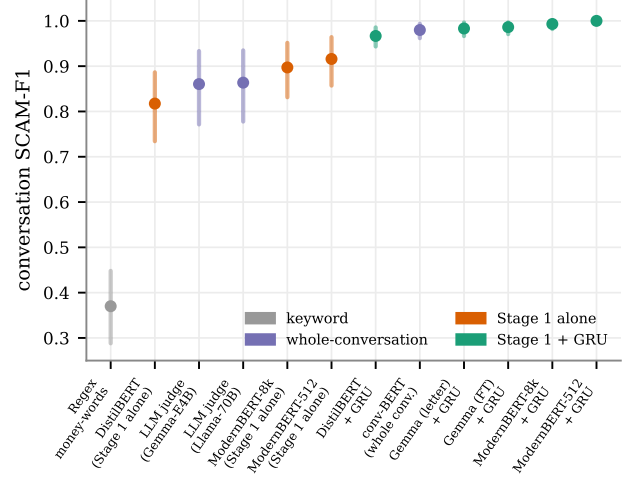


Figure 7: Every baseline and system configuration ranked by held-out conversation SCAM-F1 (dot = estimate; line = bootstrap 95% CI from 5,000 multinomial resamples of the confusion counts; multi-seed systems pool three seeds).

RQ2 takeaway: trajectory reasoning

Reasoning over the trigger sequence outperforms every alternative on the same held-out split: keyword matching (0.37), LLM judges reading the complete transcript (0.86–0.92), per-message any-demand aggregation (0.82–0.92), and the whole-conversation fine-tune (0.98) all sit at or below the two-stage systems (0.97–1.00), and none of them can produce a verdict while the conversation is still unfolding.

Among causal Stage 2 architectures at parameter parity, the GRU and LSTM are statistically indistinguishable and both outperform a causal Transformer by 0.012–0.025 test SCAM-F1; we adopt the GRU for its smaller checkpoint (2.3 vs 3.1 MiB). Full error counts, the architecture comparison, and the Stage 1-only-at-its-own-optimum control are reported in Appendix H.

Adversarial padding robustness. A scammer who knows a context-window detector is reading the conversation can pad around the scam-bearing messages with harmless filler, corrupting the window contents the model reads. We evaluate the strongest placement of this attack: $b \in \{1, \dots, 10\}$ benign distractors inserted immediately before the first scam-indicative message of every test SCAM conversation, with the full pipeline replayed on the padded conversations. End-to-end SCAM recall is essentially unchanged at every budget (flat on two picks, within 0.007 on the third; fewer than 1.3% of trigger predictions change), because the GRU integrates evidence over the whole conversation and the verdict-critical Financial_Demand messages are the trigger class least dependent on context. The full attack table is in Appendix H.

4.3.6 Warning Lead Time

RQ4 asks whether the detector can warn early enough to interrupt the victim’s engagement before money is requested. Because Stage 2 is unidirectional, its trained heads apply at every timestep, so we stream each held-out conversation message by message, exactly as the on-device engine runs, and record when the setup-warning head first crosses $\tau = 0.5$ (the boundary that binary cross-entropy training centres by construction). A scam conversation’s *lead* is the number of messages between that first warning and the conversation’s first money request (its first annotated Financial_Demand message); a positive lead means the user was warned before the money ask. Table 7 reports the results for the end-to-end picks, averaged over the three training seeds.

At every pick the median scam conversation is flagged several messages before the first financial demand and the majority are warned at or before the money ask, while the SCAM head fires on almost no benign conversations, so the warning functions as a soft caution on top of a low-false-alarm verdict. Figure 8 shows the per-conversation leads for the deployed configuration; the late warnings concentrate in short conversations whose money request is the first suspicious message at all.

A deployment can also raise or lower the warning threshold without retraining, warning on more scam conversations at the cost of more false alarms, or the reverse; all numbers above use the default $\tau = 0.5$.

RQ4 takeaway: warning lead

The auxiliary setup-warning head fires before the first money request in the majority of scam conversations, many messages ahead of it, while the terminal SCAM verdict retains a near-zero benign false-alarm rate (Table 7); the late or missing warnings concentrate in short conversations whose money request is the first suspicious message at all.

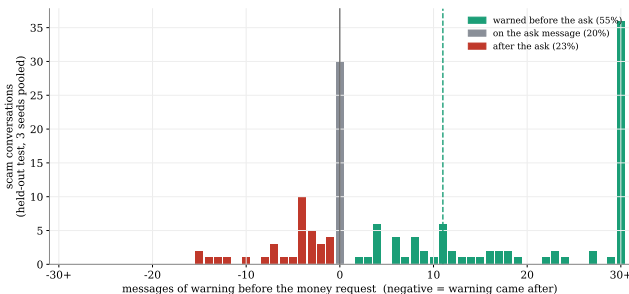


Figure 8: Warning lead per scam conversation for the deployed configuration (three seeds pooled): messages between the first setup warning and the first money request; negative values mean the warning came after the request.

Table 7: Warning lead time at the end-to-end picks ($\tau = 0.5$, mean over three seeds); FAR columns are the fractions of benign conversations on which each head ever fires.

Encoder + GRU	k	Lead (msgs)	Warned early	1st alert	SUSP FAR	SCAM FAR
DistilBERT 512	1	6.5	75%	msg 3	0.080	0.002
ModernBERT-SDPA 512	5	5.0	66%	msg 3	0.046	0.002
ModernBERT-SDPA 8k	10	6.5	71%	msg 4	0.067	0.000

4.4 Generalization Across Held-Out Sources

The random held-out test split answers whether the model generalizes to unseen conversations drawn from the same mixture of sources. LODO asks a harder question: whether the model transfers when an entire source distribution is absent from training. Figure 9 summarizes the deployment-relevant LODO cells; the strongest completed cell per held-out source is tabulated in Appendix F. For each row, the training data are all other non-synthetic sources; the named source is used only for testing.

First, generalization to *benign* held-out sources is clean: trained without ever seeing Topical-Chat or DailyDialog, the deployment cell raises *zero* false positives on either (0 of 8,623 and 0 of 3,089 NON_SCAM conversations). Everyday interpersonal and knowledge-grounded dialogue is not mistaken for scam behaviour even out of distribution. Second, transfer to the real-victim distribution is strong: held out entirely, Love-Fraud reaches 0.903 SCAM-F1 (42 of 46 scams detected), the best evidence that a model trained largely on scammer-vs-scambaiter data still recognises genuine romance-scam trajectories. Third, Scambait is the hardest held-out source (0.569 SCAM-F1): scambait conversations are adversarial and noisy, and many never reach an explicit financial demand, so a model never trained on them under-detects and over-flags. The longer-context ModernBERT-SDPA 8k $k = 3$ cell raises Scambait recall to 0.69 but at a much higher false-positive cost, confirming Scambait as the dominant source-shift challenge. The Scambait result also reflects a training-distribution constraint: 438 of the dataset’s 484 confirmed-SCAM conversations originate from Scambait, so the Scambait-holdout

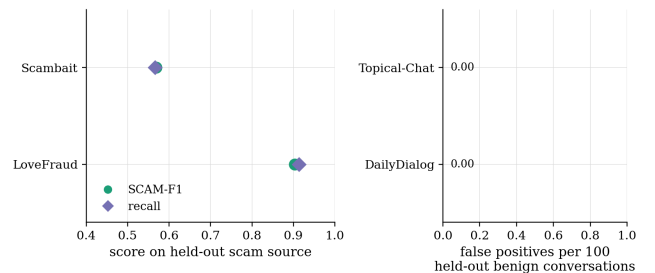


Figure 9: Leave-one-source-out generalization (ModernBERT-SDPA 512 $k = 5$). Left: SCAM-F1 and recall on held-out scam sources; right: false positives per 100 conversations on held-out benign sources.

LODO removes roughly 90% of the SCAM-positive training signal and forces the detector to learn from only the 46 Love-Fraud SCAM trajectories. The 0.574 floor should be read with that constraint in view rather than as a generic failure of the architecture.

What the test-set false positives are. The 10 false positives produced by the deployment cell on the held-out test split do not contradict the zero-FP LODO results, because the populations differ. Nine of the ten are Scambait conversations that the annotator judged never to reach terminal exploitation, the conversation is scammer-driven (it sits in the scam-source dataset) but did not, in the annotator’s reading, fully reach exploitation. The model fires SCAM on these with near-unit probability because their linguistic surface is indistinguishable from confirmed scams in the same source. Only one of the ten is a true pure-benign mistake: a DailyDialog taxi-fare exchange whose surface money-handling language and short urgent turns coincide with several trigger patterns. The zero-FP LODO results report performance on unseen *pure-benign* sources (DailyDialog, Topical-Chat in their entirety); the test-set false positives report performance on the random-split test partition in which scam-source setup-only conversations are still present.

The excluded setup-only conversations. The exclusion rule of §3.2 removed 665 scam-source conversations that exhibit the scam trajectory but end before any financial demand. These conversations belong to no training, validation, or test split, and they are the hardest negatives the dataset contains: real scammer behaviour whose ground-truth verdict is genuinely ambiguous because exploitation never materialises in the transcript. We therefore use them as an out-of-distribution stress set for the early-warning channel: the trained system streams each conversation exactly as in deployment, and we record whether and when the warning fires.

Across the three training seeds, the early-warning head fires on 62–66% of these conversations, at a median first-alert position of message 17–19, a median of 11–14 messages after the conversation’s first setup trigger. The SCAM head crosses its threshold on the same 62–66%, which on these demand-free conversations constitutes over-commitment; the warning, not the terminal verdict, is the appropriate response to a trajectory that never completes. The detector’s behavioural signal thus transfers to scam conversations it never trained on, in the majority of the hardest cases available; the roughly one third that produce no alert are dominated by short transcripts whose setup behaviour is sparse, the same regime that produces the late warnings in §4.3.6.

Generalization takeaway

Under leave-one-dataset-out evaluation the detector produces *zero* false positives on unseen benign dialogue and transfers to held-out real-victim romance-scam conversations at 0.90 SCAM-F1. The residual difficulty is

adversarial scam-bait conversations, where many interactions never reach an explicit demand; this, not benign false alarms, is the main source-shift gap.

Distributional composition of the SCAM training signal.

Of the 484 conversations carrying a confirmed SCAM label in the dataset, 438 originate from Reddit Scambait and 46 from LoveFraud. Approximately 90% of the SCAM-positive training signal therefore comes from scammer-vs-scambaiter interactions rather than scammer-vs-real-victim interactions. Because scambaiters deliberately extend conversations and respond differently from genuine victims, victim-side trigger distributions (particularly Affectionate_Language, Rapport_SelfDisclosure, and Love_Bombing receptions) are under-represented during training. This distributional bias is a stated limitation (Section 6.1); the LoveFraud-holdout LODO result, where SCAM-F1 reaches 0.915, is the strongest available evidence that the model still transfers to the real-victim distribution despite the imbalance.

5 Model Selection and Deployment Evaluation

The preceding experiments determine the model choice: every alternative family (keyword regex, per-message Financial_Demand classifiers, full-conversation conv-BERT, LLM judges, LSTM and causal-Transformer sequence reasoners) is dominated by the two-stage encoder + GRU picks, whether the encoder is a BERT-family model or the LLM adapter, on the same held-out test fold (§4). This section translates those empirical decisions into deployment profiles and implementation constraints.

5.1 Model Size and Inference Cost

The DistilBERT Stage 1 checkpoint has 66.4M parameters and is 253 MiB, while the ModernBERT-SDPA Stage 1 check-

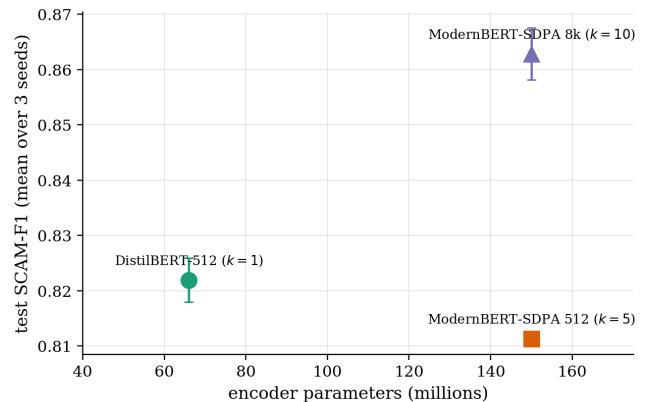


Figure 10: Encoder size versus validation SCAM-F1 after Stage 2.

point has 149.0M parameters and is 568 MiB; the accuracy gain from DistilBERT to ModernBERT-SDPA is therefore a deployment tradeoff. The LLM-adapter pick ships no encoder at all: the base model already resides on the device, and the detachable adapter plus head total 626 MiB at full precision, roughly half that at the shipped FP16. GRU is the selected sequence model because it is both smaller than LSTM (0.604M versus 0.805M parameters) and stronger on the selected SDPA cells. We report all four picks (Table 6): DistilBERT $k=1$ + GRU as the smallest and fastest, ModernBERT-SDPA 512 $k=5$ + GRU and ModernBERT-SDPA 8k $k=10$ + GRU as the statistically inseparable top of the ranking, and the LLM adapter $k=5$ + GRU, whose encoder rides the model the device already carries.

Measured on-device behaviour. We validate the full pipeline end-to-end by bulk-replaying the held-out test conversations through the integrated Element X Android client (757 conversations replayed message-by-message through the on-device engine) and diffing the on-device predictions against the server-side checkpoint. The isolation results attribute each potential source of divergence separately: the ONNX export is lossless (100% trigger agreement against the PyTorch FP32 checkpoint); the on-device WordPiece tokenizer reproduces the HuggingFace tokenizer on 98% of messages; and dynamic INT8 quantization is the entire remaining gap, reducing on-device SCAM recall from 0.804 to 0.686 on the replayed split. Mean on-device latency per suspect message is 435 ms, within interactive-messaging budgets. Because the export and tokenizer steps are exact, the INT8 gap is eliminated by precision choice rather than retraining: the shipped FP16 LiteRT export agrees with the trained checkpoint on 100% of predicted triggers for both encoder families, and the per-timestep Stage 2 export is exactly equivalent to the dynamic-length reference at every prefix length (§3.8).

5.2 Privacy-Preserving Deployment

The safest deployment strategy is on-device inference. In an open-source messaging application, each new incoming or outgoing message is processed locally by Stage 1. Stage 1 emits a trigger-probability vector and optional trigger label; the application stores this vector in a local evidence buffer rather than storing or transmitting the raw message text for model inference. Stage 2 then reads the accumulated vector sequence and updates the conversation-level risk locally after each message. No raw message text leaves the user’s device in this deployment mode. If server-side support is necessary, the privacy-preserving variant is a hybrid design in which Stage 1 remains local and only trigger sequences or aggregate risk scores are transmitted. An API that receives raw chat text is a weaker privacy deployment and applies only to explicitly consented or enterprise-controlled settings.

Figure 4 shows the local evidence trace (per-message table in Appendix F) corresponding to the running example

in Figure 1. The phone UI displays ordinary messages; internally, the application stores trigger probabilities and state updates. The same trace explains why the setup warning and the SCAM verdict are distinct: setup behavior can justify a caution before the model has evidence for a payment-demand warning.

This design also gives product-level flexibility. A soft warning can tolerate more false positives because it merely asks the user to slow down or verify the situation. A blocking or high-salience warning requires a much lower false-positive rate. Under the current test prevalence, ModernBERT-SDPA 8k $k=10$ + GRU produces approximately 6.7 false positives and 3.9 missed scams per 1,000 conversations; ModernBERT-SDPA 512 $k=5$ + GRU produces approximately 10.8 and 4.4; DistilBERT $k=1$ + GRU produces approximately 7.2 and 6.2 (error counts averaged over the three seeds; per-1,000-conversation visualisation in Appendix F).

Because real-world scam prevalence is likely lower than the test-set prevalence, false positives can dominate user experience in broad consumer deployment; the resulting pick trade-offs are summarised in §6.

RQ5 takeaway: the cost of on-device privacy

Keeping raw text on the device costs no accuracy: the shipped FP16 pipeline reproduces the trained model’s trigger predictions exactly under bulk replay in a production messaging client, INT8 quantization, the sole measurable accuracy cost, is not shipped, and the remaining price of privacy is 435 ms of on-device latency per suspect message.

6 Discussion

Why trajectory reasoning wins. Demand-shaped evidence is only interpretable inside a trajectory. Stage 2’s gain over every per-message and whole-conversation alternative comes from weighing Financial_Demand evidence against the setup behaviour that preceded it, which is why its largest effect is false-positive control rather than raw recall, and why systems that read the same text without the trajectory structure plateau below it. The same mechanism serves the demand-free region: real victim conversations that end before any demand are invisible to a demand-triggered detector, and the setup-warning channel is what surfaces them.

What near-ceiling accuracy does and does not mean. Held-out SCAM-F1 of 0.97–1.00 indicates that this benchmark is close to saturated, not that the problem is solved: the exclusion of demand-free scam approaches removes the hardest negatives, and the benign sources differ from the scam sources in topic as well as in behaviour. The claim therefore rests on the harder evaluations, the excluded setup-only conversations, the leave-one-dataset-out transfers, and the distractor-padding ablation, where errors reappear and separate systems that the

saturated split cannot.

Base rates and the cost of a warning. Our test prevalence (3.9% SCAM) is far above what a deployed detector will face, and at realistic prevalence even a small false-positive rate implies that most warnings shown on benign conversations are false. This is why the system separates a soft setup caution from the SCAM verdict: the caution is cheap to show and cheap to dismiss, the verdict is reserved for demand-stage evidence with a near-zero false-alarm rate, and either threshold can be calibrated to the deployment’s tolerance without retraining.

What near-ceiling accuracy does and does not mean. Held-out SCAM-F1 of 0.97–1.00 indicates that this benchmark is close to saturated, not that the problem is solved: the exclusion of demand-free scam approaches removes the hardest negatives, and the benign sources differ from the scam sources in topic as well as behaviour. The claim therefore rests on the harder evaluations, the excluded setup-only conversations, the leave-one-dataset-out transfers, and the distractor-padding ablation, where errors reappear and separate systems that the saturated split cannot.

Base rates and the cost of a warning. Our test prevalence (3.9% SCAM) is far above what a deployed detector will face, and at realistic prevalence even small false-positive rates mean that most warnings shown on benign conversations are false. This is why the system separates the soft setup caution from the SCAM verdict: the caution is cheap to show and cheap to dismiss, the verdict is reserved for demand-stage evidence with a near-zero false-alarm rate, and each threshold can be calibrated to a deployment’s tolerance without retraining.

What the LLM results imply. The same model spans complete failure and state-of-the-art competitiveness depending only on whether it is trained for the task, which cautions against the growing practice of deploying prompted LLMs as scam detectors [41, 52] and supports hybrid designs in which the LLM contributes representations or supervision while compact discriminative models make the decisions. For deployment it means the device’s resident model can absorb the encoder role, so the marginal footprint of scam detection falls to an adapter and a small recurrent head.

6.1 Limitations

Scambait distribution shift and SCAM-class composition. Approximately 90% of the SCAM-labeled training signal (422 of 469 active SCAM conversations) originates from Reddit Scambait, where the target side is a scambaiter who deliberately extends and provokes the conversation rather than a real victim. Victim-side trigger distributions are therefore under-represented during training. The LoveFraud LODO row demonstrates that the trained model still transfers to the real-victim distribution (SCAM-F1 0.903), but generalization claims to authentic victim populations rest on this single 46-SCAM holdout. Increasing the volume of real victim

transcripts is the most important remaining data-collection priority.

Limited real victim data. Our real victim transcripts are limited to 83 LoveFraud conversations, with 46 reaching EXPLOITATION (the SCAM label) and 36 ending in the pre-exploitation setup. Scambait provides real scammer behavior at scale, but the limited number of authentic victim transcripts constrains generalization claims.

Random-split vs source-holdout gap. Random-split test FPR is approximately 1.1% for the ModernBERT-SDPA 512 $k=5$ pick; holding out the Scambait source entirely raises the false-positive rate on that source to 25.5% (Appendix F). This gap shows that source-holdout evaluation is the relevant measure for distribution shift, while the random split measures in-distribution performance.

Silver labels. LLM annotations are not a human gold standard. We measure inter-LLM agreement across three architecturally diverse 70B-class annotators (Section 3.3) and define a separate human-validation protocol for the silver labels. The headline classifier metrics therefore evaluate agreement with the LLM-derived silver labels.

Benign financial-demand coverage. Financial_Demand is applied to any explicit money request regardless of context. The dataset therefore contains benign Financial_Demand exemplars from the casual-dialogue sources (utility bills, restaurant tabs, friends asking for money), and Stage 2 has both benign-context and scam-context demand examples to learn from. Because scam-context demand examples are more common in the active dataset than benign-context demand examples, we report false-positive counts separately for benign-source holdouts and treat benign financial language as a primary evaluation axis.

English only. All datasets are English-language. Pig-butcher operations increasingly target non-English-speaking populations, and generalization to other languages is untested.

Adversarial robustness. We evaluate one attack class directly, benign-distractor padding before the first setup trigger, and find end-to-end recall unchanged up to 10 inserted distractors on the completed picks (§4). Scammers who learn the trigger taxonomy could instead adapt their language to avoid triggering specific behavioural patterns (paraphrasing the demand, spreading it across turns); robustness to linguistic adaptation is a next step.

Deployment benchmarking. On-device latency, export equivalence, and the INT8 quantization gap are measured for the DistilBERT profile (§5); the ModernBERT-SDPA profiles have not yet been benchmarked on-device, so whether the larger ModernBERT-SDPA operating points are feasible on commodity hardware, and their battery cost, is a next step.

7 Related Work

7.1 Pig-Butchering and Romance-Fraud Studies

Pig-butchering scams have only recently received systematic empirical study. Acharya and Holz [1] analyse pig-butchering across four social media platforms, public abuse reports, and news articles, organising scammer behaviour into four broad categories (initial contact, relationship-building, scammer techniques, emotional manipulation) and confirming that the trust-building phase typically spans 3–8 weeks before any financial extraction. Oak and Shafiq [46] contribute the first qualitative analysis from in-depth victim interviews ($N=26$), identifying a finer seven-stage lifecycle (Lure, Bond, Bait, Feed, Squeeze, Cut, Encore) and emphasising that the most psychologically manipulative phase precedes any explicit financial ask. Spokoyny et al. [51] introduce CHATTERBOX, an LLM-based honey-persona system that engages live scammers and observes a three-phase progression (inbound + cross-platform request, bait + grooming, cash-out escalation); their data complements victim-side accounts with attacker-side trajectories. Whitty [55] and Buchanan and Whitty [6] provide foundational analyses of online romance-scam victimisation but predate the cryptocurrency-investment variant central to modern pig-butchering. Our work consolidates the lifecycle descriptions of these four studies into a single deterministic state machine (Section 3.1) and operationalises it as supervision for a streaming classifier; Appendix A reports the explicit mapping between our five states and the stage definitions used by Acharya, Oak, and Spokoyny.

7.2 Conversational Scam Detection

Prior detection work spans keyword matching, message-level classification, conversation-level classification, and conversational reasoning. Cross [12] surveys keyword and rule-based fraud detectors and reports their characteristic high recall / low precision profile because legitimate conversations also contain financial vocabulary. Per-message classifiers fine-tuned on individual messages [14, 39] improve precision through learned semantic features but discard the conversational context that distinguishes scam-indicative behaviour from benign messages with similar surface form. Bag-of-words and TF-IDF SVM approaches [47] classify whole conversations but cannot localise which message triggered the alert and therefore cannot measure detection latency. Gao et al. [19] propose a hierarchical dual-role network for telephone fraud that captures temporal patterns and inspires our use of a sequence reasoner over per-message trigger probabilities. Most recently, Chang and Aïmeur [8] use LLMs to identify scam indicators in chat segments, demonstrating that contextual reasoning over a conversation prefix is feasible with current-generation models; our work moves the contextual reasoning into the

labelling pipeline (Section 3.3) and uses a much smaller distilled classifier at inference time so that the system can run on user devices. The limits of LLMs as *end-user* scam detectors are also instructive: evaluating four off-the-shelf models (GPT-5, LLaMA 3.3, LLaMA 4 Scout, and a fraud-tuned Mistral) under the zero-shot prompting an ordinary user would attempt, Lu et al. [41] report false-positive rates as high as 60% and false-negative rates up to 18.8%, with the worst accuracy precisely on the reply-based romance and wrong-number conversations central to our setting. Topcuoglu et al. [52] provide the broadest such assessment to date, benchmarking nine LLMs across prompting strategies on 2,742 single-item scam samples: the best models reach only $\sim 65\%$ micro-F1, while a fine-tuned BERT trails the top LLMs and degrades on unseen scam types, suggesting pre-trained knowledge aids generalization. Our results refine this picture for the conversational setting: prompting alone fails at the fine-grained behavioural task even for a capable model, whereas a small trained adapter on the same model matches dedicated fine-tuned encoders (§4), and the resulting detector operates message by message rather than on a completed sample. Topcuoglu et al. conclude that optimal performance lies not in model size but in hybrid designs that pair the semantic understanding of LLMs with the precision and efficiency of compact classifiers; SIRENBERT is such a design, using the LLM for semantic supervision (annotation) or as an adapted feature extractor, while compact discriminative models perform the deterministic per-message and sequence classification. These findings motivate two design choices: we do not rely on zero-shot LLM judgement at inference time, and we use the LLM only as a context-conditioned, schema-constrained annotator (Section 3.3) or as a trained Stage-1 encoder, never as a prompted detector.

7.3 LLM-Based Data Annotation

Using large language models as annotators for downstream-task training has become an established methodology. Gilardi et al. [22] show that ChatGPT outperforms crowd workers on several text-annotation tasks; He et al. [28] formalise the AnnoLLM framework with self-generated annotation guidelines. These works annotate static documents (tweets, news articles, survey responses) where the entire text is visible to the annotator at once. Our setting differs in two critical ways. First, we impose a sequential constraint: the LLM annotator sees only messages $m_1, \dots, m_t$ when labelling m_t , mirroring the information available to a deployed streaming detector and eliminating future-message leakage. Second, our source datasets carry only conversation-level labels, so we use the LLM to manufacture per-message behavioural supervision under that sequential constraint rather than to summarise existing annotations. To our knowledge SIRENBERT is the first system to apply LLM-as-annotator to scam-trajectory labelling, with each label produced from the conversation prefix alone.

8 Conclusion

We presented SIRENBERT, an on-device system that detects pig-butcherer as the conversation unfolds: an encoder classifies each message into a behavioural trigger, and a recurrent model reasons over the resulting trajectory to re-evaluate risk after every message. Trained on a new message-level dataset of 12,296 conversations, SIRENBERT outperforms keyword, per-message, whole-conversation, and LLM-judge baselines, warns before the first money request in the majority of scam conversations, and runs inside a production Android messaging client with no message content leaving the device. The results support a simple conclusion: what makes pig-butcherer detectable in time to matter is not the demand, but the behavioural trajectory that precedes it.

Ethics Considerations

This work uses scam and benign conversational data and trains a detector intended to protect potential victims. We take the following steps to minimise harm and uphold responsible-research practice.

Data provenance and PII. The Reddit r/scambait dataset is drawn from publicly posted screenshots that users shared to expose and document scammers, collected in accordance with Reddit’s Developer Platform Guidelines and Public Content Policy; the LoveFraud transcripts are victim-reported and were already de-identified by the source community. Our OCR extraction retains only the message text relevant to classification: no usernames, profile metadata, or contact details are captured or stored, and we do not attempt to re-identify any individual. Human annotators were shown only the minimal message content required for labelling, with no surrounding account or profile information. Benign sources (Topical-Chat, XDailyDialog) are established public research datasets. Following guidance for research on publicly posted user content [41, 52], we treat the data as observational and avoid user-level profiling.

Victim-derived content. Romance-scam transcripts can contain sensitive personal disclosures by victims. We use them only to learn scammer behavioural patterns, never reproduce full transcripts or identifying detail in the paper, and paraphrase or synthesise any illustrative examples.

Dual-use. A scam-trajectory model could in principle inform an adversary refining their scripts. We judge the defensive benefit to outweigh this risk: the behavioural stages we model are already documented in prior victim studies, and the system reveals no new attack technique. We release trained detector artefacts and evaluation code but not raw victim transcripts.

Deployment harms. False positives can disrupt legitimate relationships and false negatives can give false assurance. We therefore (i) report false-positive and false-negative behaviour explicitly rather than headline accuracy alone, (ii) surface a lower-confidence setup warning rather than forcing a binary

scam call, and (iii) run entirely on-device so that no chat content is transmitted or retained off the user’s phone.

Open Science

In compliance with the Open Science policy, we will share the research artifacts of this work: the per-message trigger annotations and splits, the annotation prompts and framework, all training and evaluation code, the trained model checkpoints, and the Android integration code. Raw victim-derived conversation text (LoveFraud and Scambait) is excluded from release in any form, including reconstruction scripts, consistent with our ethics commitments; annotations for these sources are keyed to identifiers so that researchers who obtain the underlying text from the original sources can align them, and the benign datasets are already publicly available under their own licences. Artifacts are available anonymously to reviewers during submission and will be archived in a persistent repository upon acceptance.

References

- [1] Bhupendra Acharya and Thorsten Holz. An explorative study of pig butchering scams. *arXiv preprint arXiv:2412.15423*, 2024.
- [2] Theodore W. Anderson and Leo A. Goodman. Statistical inference about markov chains. *The Annals of Mathematical Statistics*, 28(1):89–110, 1957.
- [3] Dogu Araci. FinBERT: Financial sentiment analysis with pre-trained language models. *arXiv preprint arXiv:1908.10063*, 2019.
- [4] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1):289–300, 1995.
- [5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [6] Tom Buchanan and Monica T. Whitty. Online dating romance fraud: The victims and the scams. *Journal of Financial Crime*, 19(2):168–184, 2012.
- [7] Gavin C. Cawley and Nicola L. C. Talbot. On overfitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11(70):2079–2107, 2010.
- [8] Yuan-Chen Chang and Esma Aïmeur. Chat or trap? detecting scams in messaging applications with large

- language models. In *Proceedings of the 8th Cyber Security in Networking Conference (CSNet)*, pages 92–99, 2024.
- [9] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.
- [10] Robert B. Cialdini. *Influence: Science and Practice*. Allyn & Bacon, 4th edition, 2001.
- [11] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.
- [12] Cassandra Cross. Fraud recovery and the normalization of victimisation. *Trends and Issues in Crime and Criminal Justice*, (608):1–14, 2020.
- [13] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.
- [14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pages 4171–4186, 2019.
- [15] Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah A. Smith. Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping. *arXiv preprint arXiv:2002.06305*, 2020.
- [16] Jesse Dodge, Maarten Sap, Ana Marasović, William Agnew, Gabriel Ilharco, Dirk Groeneveld, Margaret Mitchell, and Matt Gardner. Documenting large webtext corpora: A case study on the Colossal Clean Crawled Corpus. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1286–1305, 2021.
- [17] Matthew Edwards, Guillermo Suarez-Tangil, Haitao Peng, Awais Rashid, Gianluca Stringhini, and Lorenzo Cavallaro. An explorative study into the romance scam: An analysis of the conversations of online romance scams. In *Proceedings of the IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 1–8. IEEE, 2018.
- [18] Federal Bureau of Investigation. Internet crime report 2023. Technical report, Internet Crime Complaint Center (IC3), 2024.
- [19] Jianhua Gao, Jian Wang, Yang Zhang, and Pan Li. Hierarchical dual-role interaction network for real-time telephone fraud detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- [20] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020.
- [21] Gemma Team. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.
- [22] Fabrizio Gilardi, Meysam Alizadeh, and Maoel Kubli. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30):e2305016120, 2023.
- [23] Karthik Gopalakrishnan, Behnam Hedayatnia, Qinlang Chen, Anna Gottardi, Sanjeev Kwatra, Anu Venkatesh, Raefer Gabriel, and Dilek Hakkani-Tür. Topical-chat: Towards knowledge-grounded open-domain conversations. In *Proceedings of INTERSPEECH*, pages 1891–1895, 2019.
- [24] Kyle Gorman and Steven Bedrick. We need to talk about standard splits. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 2786–2791, 2019.
- [25] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [26] John M. Griffin and Kevin Mei. How do crypto flows finance slavery? The economics of pig butchering. *SSRN Working Paper 4742235*, 2024.
- [27] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
- [28] Xingwei He, Zhenghao Lin, Yeyun Gong, A.-Long Jin, Hang Zhang, Chen Lin, Jian Jiao, Siu Ming Yiu, Nan Duan, and Weizhu Chen. AnnoLLM: Making large language models to be better crowdsourced annotators. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.
- [29] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

- [30] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [31] Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.
- [32] Edward L. Kaplan and Paul Meier. Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282):457–481, 1958.
- [33] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [34] Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*, volume 2, pages 1137–1143, 1995.
- [35] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PageAttention. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.
- [36] J. Richard Landis and Gary G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174, 1977.
- [37] Yanran Li, Hui Su, Xiaoyu Shen, Wenjie Li, Ziqiang Cao, and Shuzi Niu. DailyDialog: A manually labelled multi-turn dialogue dataset. In *Proceedings of the International Joint Conference on Natural Language Processing (IJCNLP)*, pages 986–995, 2017.
- [38] Jianhua Lin. Divergence measures based on the shannon entropy. *IEEE Transactions on Information Theory*, 37(1):145–151, 1991.
- [39] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- [40] LoveFraud.com. LoveFraud02: Romance scam victim transcript collection, 2024. 83 victim-reported romance scam transcripts collected from the LoveFraud.com support community.
- [41] Allison Lu, Bernardo B. P. Medeiros, Kevin R. B. Butler, and Patrick Traynor. Read this paper to get \$50 million: An analysis of mobile messaging scams using reddit data. *arXiv preprint arXiv:2605.16656*, 2026.
- [42] Jonathan Lusthaus, Yueran Luo, and Thomas J. Holt. Pig butchering (Sha Zhu Pan): A modern romance scam evolved. *Trends in Organized Crime*, 2023.
- [43] Nathan Mantel. Evaluation of survival data and two new rank order statistics arising in its consideration. *Cancer Chemotherapy Reports*, 50(3):163–170, 1966.
- [44] R. Thomas McCoy, Ellie Pavlick, and Tal Linzen. Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 3428–3448, 2019.
- [45] Marius Mosbach, Maksym Andriushchenko, and Dietrich Klakow. On the stability of fine-tuning BERT: Misconceptions, explanations, and strong baselines. In *Proceedings of the 9th International Conference on Learning Representations (ICLR)*, 2021.
- [46] Rajvardhan Oak and Zubair Shafiq. “Hello, is this Anna?”: Unpacking the lifecycle of pig-butchering scams. In *USENIX Symposium on Usable Privacy and Security (SOUPS)*, 2025.
- [47] Elaheh Raisi and Bert Huang. Text mining for automatic detection of online predatory conversations. *Online Social Networks and Media*, 12:33–42, 2019.
- [48] Sebastian Raschka. Model evaluation, model selection, and algorithm selection in machine learning. *arXiv preprint arXiv:1811.12808*, 2018.
- [49] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- [50] Ray Smith. An overview of the tesseract ocr engine. In *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR)*, volume 2, pages 629–633, 2007.
- [51] Daniel Spokoyny, Nikolai Vogler, Xin Gao, Tianyi Zheng, Yufei Weng, Jonghyun Park, Jiajun Jiao, Geoffrey M. Voelker, Stefan Savage, and Taylor Berg-Kirkpatrick. Victim as a service: Designing a system for engaging with interactive scammers. *arXiv preprint arXiv:2510.23927*, 2025.
- [52] Cem Topcuoglu, Seyed Ali Akhavan, Harel Berger, Sadia Afroz, Michalis Pachilakis, Vibhor Sehgal, Leyla Bilge, and Engin Kirda. Measuring and evaluating the

performance of generative AI models for scam detection. *arXiv preprint arXiv:2607.17353*, 2026.

- [53] Sudhir Varma and Richard Simon. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7(1):91, 2006.
- [54] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Nathan Cooper, Griffin Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *arXiv preprint arXiv:2412.13663*, 2024.
- [55] Monica T. Whitty. The scam victim: An analysis of the personality, lifestyle, and demographic characteristics of those who fall for online romance scams. *Journal of Financial Crime*, 19(2):152–167, 2012.
- [56] An Yang, Baosong Yang, Beichen Zhang, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [57] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.

A Mapping of Our Five States to Prior-Work Stage Definitions

Section 3.1 consolidates the lifecycle descriptions of three pig-butchering studies into our five-state finite-state machine. Table 8 reports the per-stage mapping. Cells are joined when a single one of our states absorbs multiple stages from the source paper, reflecting the collapse decisions explained in Section 3.1.

B Trigger Definitions and Decision Rules

The trigger taxonomy is defined in §3.1. The full annotation guide defines each trigger using a positive definition, exclusion criteria, and boundary cases. For example, Love_Bombing requires disproportionate romantic intensity or premature commitment rather than ordinary affection; Credential_Positioning requires authority, status, professional expertise, or wealth claims used to shape trust; Financial_Demand requires an unsolicited or context-inconsistent request for money, gift cards, cryptocurrency, banking action,

deposits, fees, or investment. These decision rules prevent ordinary financial conversation from being labeled as exploitation.

C Annotation Prompt

The sequential annotation prompt contains four components: the trigger taxonomy, the finite-state transition rules, the current conversation prefix, and a guided JSON schema. For message m_t , the annotator receives only messages $m_1, \dots, m_t$ and returns the trigger label, intent, state before the message, state after the message, and a short rationale. Guided decoding constrains the output to the valid trigger and state sets, yielding structured per-message labels without post-hoc parsing.

Role. You are annotating a conversation for pig-butchering fraud indicators under a real-time constraint. You may use only the messages shown in the current prefix.

Task. For the current message, assign exactly one behavioral trigger from the taxonomy and update the lifecycle state only when the current message provides sufficient evidence.

Decision rule. Do not label ordinary affection, professional self-description, or legitimate financial discussion as scam evidence unless it functions as part of a coercive, deceptive, or exploitative trajectory. Financial_Demand requires an unsolicited or context-inconsistent request for money, gift cards, cryptocurrency, banking action, deposits, fees, or investment.

Output schema. Return JSON with fields `message_id`, `speaker`, `trigger`, `state_before`, `state_after`, `risk_label`, and `rationale`.

D Per-Source Evaluation

Table 9 reports the source of Stage 2 errors for the two operating points discussed in the main body. False positives are primary NON_SCAM conversations predicted as SCAM. False negatives are true SCAM conversations predicted NON_SCAM. Long benign reference conversations are excluded from active classifier training and appear only in the reliability analysis.

The dominant false-positive source is primary NON_SCAM Scambait. This is expected to be difficult: Scambait contains real scammer behavior, but some transcripts do not reach a terminal SCAM label under the annotation policy. These are not cases where conversations without Financial_Demand are treated as scams by definition; they are precisely false positives where the model overpredicts SCAM on primary NON_SCAM Scambait conversations.

E Trajectory Divergence Analysis

The trigger-label sequence of a conversation is its behavioural trajectory; this appendix quantifies how SCAM and

Table 8: Mapping between our five FSM states (rows) and the stage definitions used by the three prior pig-butcher lifecycle studies (columns). “—” indicates a stage we deliberately omit; *Cut* (scammer disappears) and *Encore* (recovery scams from a new persona) lie outside the original chat conversation that our classifier scores.

Our state	Acharya & Holz [1]	Oak & Shafiq [46]	Spokoyny et al. [51]
NULL	(pre-contact)	(pre-Lure)	(pre-inbound)
INITIAL_CONTACT	Initial Contact (16 platforms)	Lure	Inbound
RELATIONSHIP_BUILDING	Relationship-Building (10 sub-techniques: Friendship, Romance, Trust, ...)	Bond	(within Days 0–5 grooming)
PRE_EXPLOITATION	Scammer Techniques (5) + Emotional Manipulation (5)	Bait + setup portion of Feed/Squeeze	Cross-Platform Request + Bait + Grooming
EXPLOITATION	Financial Loss / Cryptocurrency Transfer Requests / Fake Investment Platforms	Feed + Squeeze (money flow)	Cash-Out Escalation
(omitted)	—	Cut, Encore	—

Table 9: Source of false-positive and false-negative errors at the end-to-end picks (held-out test split, seed 42).

Model	FP total	FP Scambait	FN total	FN Scambait
DistilBERT k=1 + GRU	10	8	8	7
ModernBERT-SDPA 512 k=5 + GRU	15	13	5	5
ModernBERT-SDPA 8k k=10 + GRU	8	8	6	5

NON_SCAM trajectories differ across the dataset (12,296 conversations), using the annotated trigger labels.

Five standard analyses, each with its significance test, quantify the divergence visible in Figure 3. (1) *Trigger use*. A χ^2 test of independence on the trigger-by-class table rejects strongly ($\chi^2=45,975$, $p\approx 0$) with a large effect size (Cramér’s $V=0.516$); 13 of 14 triggers differ individually after Benjamini–Hochberg correction [4]. (2) *Dynamics*. Per-origin χ^2 homogeneity tests on the first-order transition matrices [2] reject for 12 of 14 origin triggers (BH $p<0.05$); the Jensen–Shannon divergence [38] between the class transition kernels is 0.149. (3) *Timing*. The Jensen–Shannon divergence between class trigger distributions, per decile of normalized conversation progress, is highest at the opening (0.539 in the first decile, falling to 0.207) and exceeds a 1,000-shuffle permutation null in all ten deciles: the classes are most distinguishable *early*. (4) *Escalation speed*. Kaplan–Meier analysis [32] of the time to the first pre-exploitation trigger, with never-escalating conversations right-censored, gives a median of 3 suspect messages for SCAM, while the median NON_SCAM conversation never escalates (log-rank [43] $\chi^2=3,740$, $p<10^{-6}$). (5) *Whole-sequence separation*. On a balanced sample, between-class Levenshtein distance exceeds within-class distance by 8.2 edits (permutation $p=0.001$), so the trajectories separate as whole sequences, not only in parts. Recomputing all five statistics on Stage 1 *predicted* trajectories reproduces the values computed on the annotated labels closely, so the deployed encoder preserves the trajectory structure it is trained on.

F Additional Generalization and Deployment Material

This appendix collects the per-source LODO table, the per-message on-device evidence trace, and the population-scale

error visualisation referenced in the main text.

G Per-Class Stability Analysis

Table 12 reports, for every trigger class, the range its F1 spans across the context windows; Figure 12 gives the full per-cell view. The table makes the pattern easy to read. Classes with large test support are stable: *SmallTalk_Maintenance* (15,270 test messages) and *Financial_Demand* (332) move by at most 0.06 across the entire sweep for every encoder. Classes with small support swing widely: *WrongNumber_Intro* (29 test messages), *Crisis_Fabrication* (73), *Investment_Pitch* (88), and *Tech_Simulation* (123) each move by up to 0.16 for the same encoder. The reason is arithmetic: with only a few dozen positive messages, reclassifying a handful of borderline cases moves that class’s F1 by several points, and different fine-tunes place their decision boundaries slightly differently near those borderline messages [15, 45]. Because these swings go in different directions for different classes, they largely cancel in the macro average; the residual adjacent- k macro-F1 differences of 1–3 points are therefore dominated by initialisation variance, and we do not interpret differences inside this band.

H Additional Stage-2 Results

This appendix reports the full Stage-2 ablation material summarised in the main text: per-split error counts (Table 13), the causal-architecture comparison at parameter parity (Table 14), the Stage 1-only aggregation control (Table 15), the adversarial-padding attack table (Table 16), and the validation/test consistency of the Stage-2 gain (Figure 13). Evaluated at the Stage 1 picks rather than the end-to-end picks, the any-FD rule reaches test SCAM-F1 of 0.743–0.786, still below the full pipeline on every encoder even though the pipeline uses less context per message.

I Per-Bucket Message-Length Breakdown

Before comparing encoders, two per-message diagnostics explain *why* context helps and *what limits it*, by decomposing

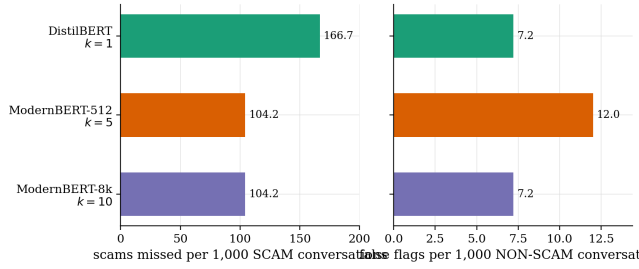


Figure 11: Expected scams missed (left) and benign conversations falsely flagged (right) per 1,000 conversations at the held-out test prevalence (3.7% SCAM), for the three deployment picks.



Figure 12: Per-class trigger F1 (test split) for every (encoder, context window k) cell. Rows are sorted top-to-bottom: DistilBERT, ModernBERT-SDPA 512, ModernBERT-SDPA 8k, each with $k \in \{0, 1, 2, 3, 4, 5, 10, 15\}$. Columns are the 14 trigger classes, ordered by test support (most frequent on the left). Cells annotated with their F1; vertical white lines separate encoder blocks.

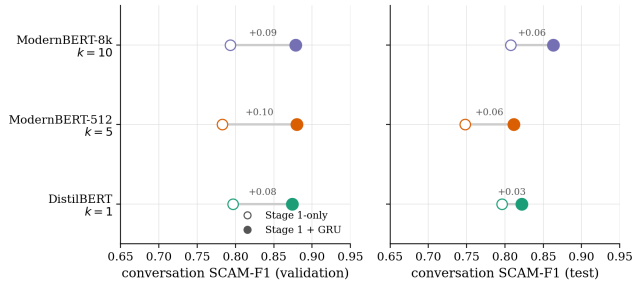


Figure 13: Conversation SCAM-F1 for the three end-to-end picks on validation (left) and test (right). Open marker: Stage 1-only any-Financial_Demand rule; filled marker: Stage 1 + GRU; dashed horizontal line: conv-BERT baseline (full-conversation binary fine-tune).

Table 10: Leave-one-source-out generalization for the deployment-oriented cell (ModernBERT-SDPA 512, $k=2$, GRU), trained on all sources except the held-out one. Scam-bearing holdouts report SCAM-F1 and TP/FN; benign holdouts report false positives against NON_SCAM conversations.

Held-out source	Cell	SCAM-F1	TP/FN	FP/TN, FPR
LoveFraud	ModernBERT-SDPA 512 $k=5$	0.903	42/4	5/32, 13.5%
Scambait	ModernBERT-SDPA 512 $k=5$	0.569	248/190	186/542, 25.5%
Topical-Chat	ModernBERT-SDPA 512 $k=5$	—	—	0/8,623, 0.0%
DailyDialog	ModernBERT-SDPA 512 $k=5$	—	—	0/3,089, 0.0%

the test set by target-message length; these diagnostics determine what we need from a second encoder. Message length is measured in characters (tokenizer-independent); bucket boundaries follow the empirical distribution and each bucket’s support N is reported in the figures.

Short messages need context; long messages can be hurt by it. Figure 14 separates the two regimes. For the shortest messages (≤ 50 characters, $N=7,742$) (“ok”, “sure”, “send it”, “okay sir”, “money”, “how are you”) trigger accuracy climbs steeply with context for all three encoders (e.g. ModernBERT from ≈ 0.76 at $k=0$ to ≈ 0.83): in isolation these utterances are not interpretable as any single trigger (“money” could be a Financial_Demand turn, a polite enquiry, or just the prior speaker’s topic) and only the surrounding turns settle the assignment. For the longest messages (> 300 characters, $N=305$) the picture inverts: accuracy peaks at a small window and then *falls* for the 512-token encoders as k grows (DistilBERT and ModernBERT-SDPA 512 collapse to ≈ 0.45 – 0.49 at $k=15$), while the 8k-token ModernBERT stays flat at ≈ 0.63 . A long message plus a long context exceeds 512 tokens, so the 512-cap encoders truncate, and because the target message sits at the end of the constructed input, truncation can remove the very message being classified. This is the mechanism behind both the aggregate context gain and the large- k plateau, and it already motivates an 8k-token encoder.

Quantifying the truncation onset. Figure 15 makes the cause explicit: the constructed input grows from a median of ≈ 17 tokens at $k=0$ to ≈ 250 at $k=15$, and the share exceeding the 512-token cap stays negligible ($< 0.2\%$) through $k=5$ before reaching $\approx 8\%$ at $k=15$. So $k=15$ is the first window at which 512-token encoders truncate real inputs at scale, which is exactly where their accuracy on long inputs collapses. Figure 16 confirms the effect resolves by input token length: at $k \leq 3$ inputs almost never exceed 512 tokens and the encoders track each other, whereas at $k=15$ the over-512 bucket (shaded) populates and the 8k-token encoder, which reads the full input, pulls ahead of the two 512-token encoders.

Figure 17 gives the full per-bucket version of the short/long-message analysis above (Figure 14): trigger accuracy versus context window k for every character-length bucket and all three encoders, on the held-out test split. The pattern is mono-

Table 11: Illustrative local deployment trace. Stage 1 classifies message-level behavior; the sequence reasoner updates conversation risk from the accumulated trigger-probability vectors.

t	Message sketch	Stage 1 trigger	FSM state	Sequence risk
1	“Hi, I think I saved the wrong number.”	WrongNumber_Intro	INITIAL_CONTACT	NON_SCAM
2	“No worries.”	SmallTalk_Maintenance	INITIAL_CONTACT	NON_SCAM
3	“What do you do for work?”	SmallTalk_Maintenance	RELATIONSHIP_BUILDING	NON_SCAM
4	“I work in healthcare.”	SmallTalk_Maintenance	RELATIONSHIP_BUILDING	NON_SCAM
5	“I run a trading team and mentor clients.”	Credential_Positioning	PRE_EXPLOITATION	setup warning
6	“That sounds interesting.”	SmallTalk_Maintenance	PRE_EXPLOITATION	setup warning
7	“I can show you my platform. Move to Telegram.”	Platform_Migration	PRE_EXPLOITATION	setup warning
8	“I am not sure.”	SmallTalk_Maintenance	PRE_EXPLOITATION	setup warning
9	“Start with \$500 today before the window closes.”	Financial_Demand	EXPLOITATION	SCAM
10	“Why today?”	SmallTalk_Maintenance	EXPLOITATION	SCAM

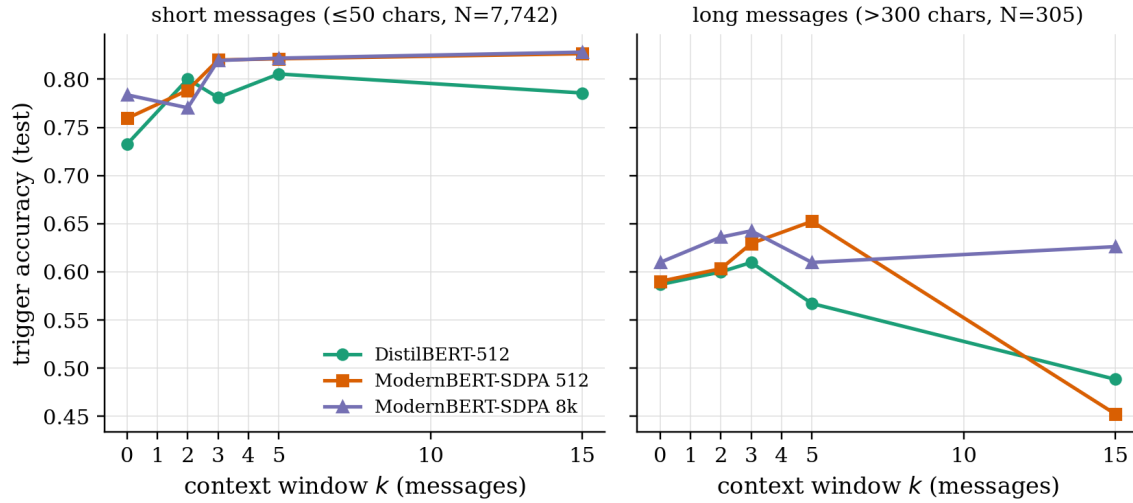


Figure 14: Trigger accuracy versus context window k for the shortest target messages (≤ 50 characters, left) and the longest (> 300 characters, right). Full per-bucket breakdown in Appendix I.

Table 12: Per-class F1 range across context windows $k \in \{0, 1, 2, 3, 4, 5, 10, 15\}$, per encoder. Classes ordered by ascending test support. The wide ranges sit on rare classes whose F1 is dominated by sample noise; Financial_Demand and SmallTalk_Maintenance are stable across k for every encoder.

Class	Test N	DistilBERT	ModernBERT-SDPA 512	ModernBERT-SDPA 8k
WrongNumber_Intro	29	[.46, .55]	[.54, .60]	[.49, .62]
Crisis_Fabrication	73	[.21, .28]	[.25, .33]	[.23, .32]
Investment_Pitch	88	[.32, .38]	[.31, .46]	[.37, .44]
Tech_Simulation	123	[.17, .25]	[.16, .27]	[.17, .33]
Urgency_Pressure	129	[.22, .29]	[.19, .31]	[.24, .36]
Platform_Migration	245	[.63, .68]	[.64, .68]	[.64, .67]
Financial_Demand	332	[.58, .64]	[.60, .66]	[.60, .64]
Love_Bombing	412	[.48, .60]	[.52, .63]	[.61, .67]
Affectionate_Language	503	[.43, .53]	[.46, .57]	[.53, .57]
Credential_Positioning	579	[.35, .39]	[.38, .45]	[.38, .43]
Emotional_Manipulation	601	[.37, .43]	[.40, .44]	[.41, .45]
Rapport_SelfDisclosure	1,020	[.39, .45]	[.40, .47]	[.41, .48]
Introduction_Opener	1,144	[.75, .89]	[.77, .89]	[.79, .89]
SmallTalk_Maintenance	15,270	[.88, .91]	[.89, .92]	[.90, .92]

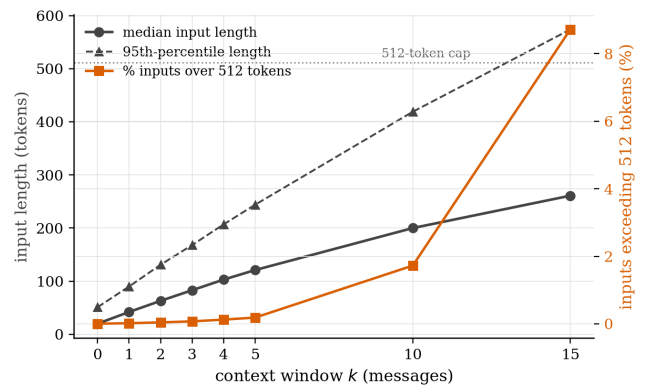


Figure 15: Constructed-input length (median and 95th percentile, left axis) and the share of inputs exceeding the 512-token cap (right axis), versus context window k .

Table 13: Stage 2 sequence-reasoner error counts on validation and test. False positives are primary NON_SCAM conversations incorrectly flagged as SCAM; false negatives are true SCAM conversations predicted NON_SCAM.

Model	Split	TP	FN	FP	TN
DistilBERT k=1 + GRU	Val	45/49	4	9	1238/1247
DistilBERT k=1 + GRU	Test	40/48	8	10	1239/1249
ModernBERT-SDPA 512 k=5 + GRU	Val	44/49	5	7	1240/1247
ModernBERT-SDPA 512 k=5 + GRU	Test	43/48	5	15	1234/1249
ModernBERT-SDPA 8k k=10 + GRU	Val	44/49	5	8	1239/1247
ModernBERT-SDPA 8k k=10 + GRU	Test	42/48	6	8	1241/1249

Table 14: Stage 2 architecture comparison at the nested-CV picks: held-out test SCAM-F1, mean $\pm$ std over seeds {7, 42, 123}, at parameter parity (hidden size 256, 2 layers).

Encoder	k	GRU	LSTM	Causal Transformer
DistilBERT 512	1	0.821 $\pm$ 0.004	0.839 $\pm$ 0.000	0.808 $\pm$ 0.018
ModernBERT-SDPA 512	5	0.810 $\pm$ 0.003	0.812 $\pm$ 0.018	0.785 $\pm$ 0.002
ModernBERT-SDPA 8k	10	0.862 $\pm$ 0.005	0.845 $\pm$ 0.024	0.845 $\pm$ 0.013

tone in message length: the gain from context shrinks as messages get longer, and the large- k collapse of the 512-token encoders is confined to the >300 -character bucket, where a long target message plus context overflows the 512-token cap.

J Model Selection Ablations

How. For each (encoder, k) configuration and each of the 5 outer folds we (i) re-fine-tune the Stage 1 encoder on that fold’s training conversations only, so the held-out fold’s trigger-probability vectors are produced by an encoder that has never seen them; (ii) train the Stage 2 GRU on the fold’s training-portion vectors, with an inner 15% stratified split for early stopping; (iii) score conversation SCAM-F1 on the held-out fold. We repeat over three random seeds (7, 42, 123) selected *a priori* – fine-tuning of pre-trained encoders is known to have substantial seed-driven variance even at fixed hyperparameters [15, 45] – and report the mean of the resulting 15 fold–seed estimates per configuration. Per encoder, the picked k is the one with the highest nested-CV mean SCAM-F1; the test split is touched exactly once, to report a single test number per pick.

Which adjacent- k differences are real. The per- k values in Table 19 (and the end-to-end grid in Table 20) differ by small amounts between adjacent k . We test every adjacent- k drop with a paired Wilcoxon signed-rank test over the 15 fold–seed estimates per cell, Bonferroni-corrected over the three metrics (macro-F1, accuracy, conversation SCAM-F1). Five of the 36 tested drops survive correction. Four align with input truncation at the 512-token cap: DistilBERT and ModernBERT-SDPA 512 both drop in trigger macro-F1 from $k=10$ to $k=15$ (0.527 $\rightarrow$ 0.503 and 0.550 $\rightarrow$ 0.533), while ModernBERT-SDPA 8k, which never truncates, rises (0.545 $\rightarrow$ 0.552). The

Table 15: Stage 1-only direct Financial_Demand aggregation versus Stage 1+GRU at the end-to-end picks, held-out test split. Stage 1-only predicts SCAM if any message has Financial_Demand as the argmax class; GRU error counts are means over the three seeds.

Model	Stage 1 FP	GRU FP	FP reduction	Stage 1 SCAM-F1	GRU SCAM-F1
DistilBERT $k=1$	14	9	33.6%	0.796	0.821
ModernBERT-SDPA 512 $k=5$	24	14	41.7%	0.748	0.810
ModernBERT-SDPA 8k $k=10$	17	9	48.8%	0.807	0.862

Table 16: Adversarial padding: end-to-end SCAM recall on the test SCAM conversations under b inserted benign distractor messages, for the full pipeline and the Stage 1-only any-FD rule (replayed on the identical padded conversations).

Pick	System	0	1	2	3	5	10
DistilBERT $k=1$	Pipeline	.833	.833	.833	.833	.833	.833
	Stage 1-only	.854	.854	.854	.854	.854	.854
ModernBERT-SDPA 512 $k=5$	Pipeline	.896	.896	.896	.896	.889	.896
	Stage 1-only	.896	.896	.896	.896	.896	.896
ModernBERT-SDPA 8k $k=10$	Pipeline	.875	.875	.875	.875	.868	.868
	Stage 1-only	.917	.917	.917	.917	.917	.917

most significant drop is in the end-to-end conversation SCAM-F1 of ModernBERT-SDPA 8k between $k=10$ and $k=15$ (Bonferroni-corrected $p=9 \times 10^{-4}$, mean drop 0.032, 6.4 additional false positives per fold), at the same k where its trigger macro-F1 is still rising; the “Picks” discussion below returns to this. All other adjacent- k movements are within fold–seed noise after correction.

How much do the two picks actually differ? The Stage 1 picks use larger k than the end-to-end picks (5 vs 1, 10 vs 5, 15 vs 10), but for most encoders this gap is not statistically meaningful. We test, for each encoder, whether the end-to-end SCAM-F1 at the end-to-end pick differs from that at the Stage 1 pick’s k , using a paired Wilcoxon signed-rank test over the 15 fold–seed estimates [13]. For DistilBERT ($k=1$ vs $k=5$) and ModernBERT-SDPA 512 ($k=5$ vs $k=10$) the difference is +0.005 SCAM-F1 and not significant ($p=0.36$ and $p=0.43$): the two picks are interchangeable, and the end-to-end curve is effectively flat over this range of k . Only for ModernBERT-SDPA 8k is the difference real ($k=10$ vs $k=15$, +0.032 SCAM-F1, $p=0.0003$). The headline is therefore not that the pipeline “prefers” less context, but that end-to-end SCAM-F1 is flat across a wide k range, so the exact context window is not load-bearing. The one significant case (ModernBERT-SDPA 8k at $k=15$) coincides with the 512-truncation regime for the other encoders and with a rise in Stage 1 mislabelling of benign messages as Financial_Demand; the recall is unchanged (0.907 vs 0.902) and the entire gap is false positives (12.8 vs 19.2 per fold), which we attribute to those mislabels, but we do not generalise it into a law over all encoders. The practical consequence is a clean selection rule: for ModernBERT-SDPA 8k the smaller window ($k=10$) is chosen because it is genuinely better (same

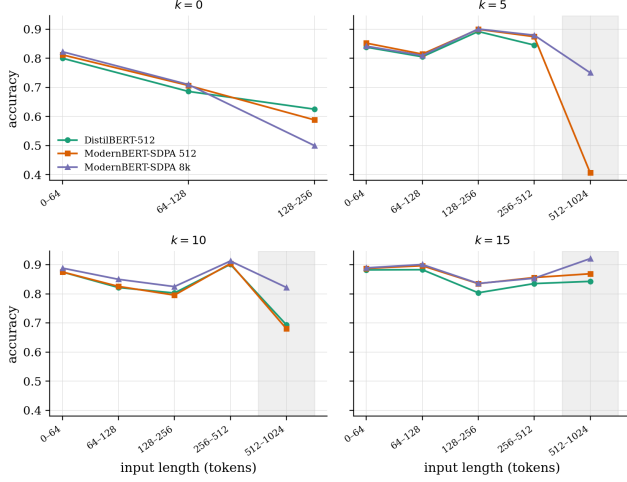


Figure 16: Accuracy by input token-length bucket, one panel per context window $k \in \{0, 2, 3, 15\}$ (over-512 bucket shaded; buckets with fewer than 30 messages omitted).

Table 17: Conversation-level stratified split; SCAM prevalence is preserved across splits.

Split	Conversations	SCAM	NON_SCAM
Train	10,368	387	9,981
Validation	1,296	49	1,247
Test	1,297	48	1,249
Total	12,961	484	12,477

recall, fewer false positives), and for the encoders where the windows are statistically interchangeable the smaller window is chosen on deployment cost (shorter constructed inputs, lower latency and memory) rather than on accuracy.

Table 20 reports the full end-to-end nested-CV grid (conversation SCAM-F1, mean over 5 folds $\times$ 3 seeds) for every encoder and context window; the Stage 1 per-message counterpart is Table 19 in the main text. The $k=0$ column is the no-context/privacy baseline: every encoder loses 0.02–0.04 SCAM-F1 relative to its pick, so context is load-bearing but the no-context system remains usable where privacy policy forbids retaining prior messages.

Token-length analysis explains why the 8k model does not dominate every setting. At $k=5$, only 0.26% of inputs exceed 512 tokens, so a 512-token encoder is usually sufficient. At $k=15$, 7.77% exceed 512 tokens, making the 8k setting meaningful.

At $k=15$, ModernBERT-SDPA 8k recovers long-input accuracy: 0.756 versus 0.620 for ModernBERT-SDPA 512 in the 512–1024 token bucket, and 0.604 versus 0.292 in the 1024–2048 token bucket. This result supports ModernBERT-SDPA 8k at $k=15$ for long-context operation, while ModernBERT-SDPA 512 at $k=5$ remains the low-false-positive deployment

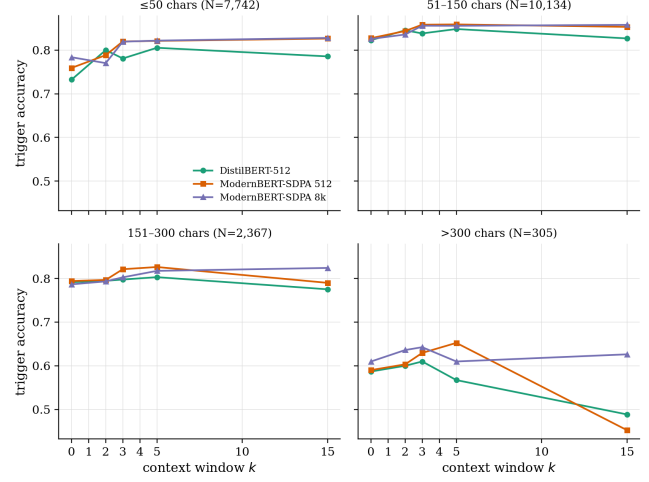


Figure 17: Trigger accuracy versus context window k for each target-message character-length bucket (test split, N per bucket shown).

Table 18: Stage 1 encoder comparison. Metrics are validation macro-F1 / accuracy for per-message 14-class trigger classification.

Encoder	k=0	k=2	k=3	k=5	k=15
DistilBERT 512	.50/.80	.53/.84	.53/.83	.53/.84	.53/.83
ModernBERT-SDPA 512	.52/.81	.55/.83	.55/.85	.55/.85	.54/.86
ModernBERT-SDPA 8k	.53/.82	.54/.82	.56/.85	.57/.85	.57/.86

configuration.

K Stage 2 Architecture Ablation

The Stage 1-only-versus-GRU comparison at the end-to-end picks and the three-way GRU/LSTM/causal-Transformer comparison are reported in Appendix H (Tables 15 and 14).

L Deployment Benchmarks

Table 22 reports checkpoint-derived model sizes. On-device latency, export equivalence, and the INT8 quantization gap for the DistilBERT profile are reported in §5; battery impact and on-device benchmarks for the ModernBERT profiles remain to be measured.

These sizes underlie the size-versus-accuracy trade-off across the three picks reported in §5.

L.1 Annotator-Side Detection Latency on LoveFraud

The annotator agreement section (§4) reports cross-LLM trigger and state agreement on the LoveFraud + benign-reference

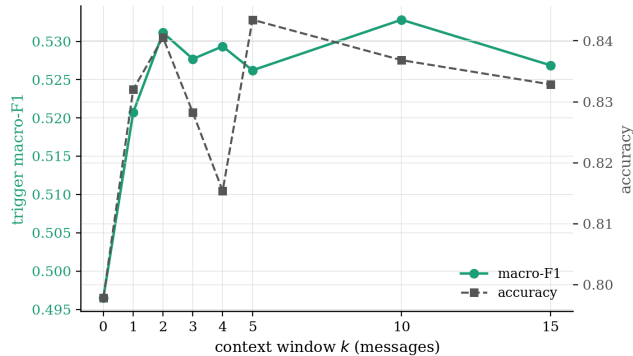


Figure 18: DistilBERT Stage 1 trigger classification by context window k (5-fold nested-CV mean $\pm$ fold std): macro-F1 (left axis) and accuracy (right axis). Dashed grey lines mark the always-majority-class baseline (predict the most-frequent trigger, SmallTalk_Maintenance, on every message); dotted grey lines mark the uniform-random baseline (each of the 14 triggers with equal probability). A learned classifier must clearly clear both.

Table 19: Stage 1 trigger macro-F1 under 5-fold nested CV (mean $\pm$ fold std). The values sit in a narrow 0.49–0.55 band; the largest within-encoder drops (DistilBERT and ModernBERT-SDPA 512 at $k=10 \rightarrow 15$) align with the 512-token truncation onset.

Encoder	$k=0$	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	$k=10$	$k=15$
DistilBERT 512	0.487	0.521	0.525	0.523	0.525	0.527	0.521	0.503
ModernBERT-SDPA 512	0.523	0.543	0.542	0.548	0.548	0.546	0.550	0.533
ModernBERT-SDPA 8k	0.506	0.526	0.541	0.538	0.547	0.545	0.545	0.552

reliability subset. A complementary timing measurement asks, on the same subset, *how early during a conversation the three annotators first call the conversation SCAM-eligible*. This is an annotator-side latency rather than a classifier-side latency: it characterises how early the per-message silver labels become available to the supervisor, not how early the trained on-device classifier fires. We report it here because it is informative for understanding the silver labels but does not bear directly on the deployed system’s detection-latency claims, which are reported on the trained classifier in §4.3.6.

With a 2-of-3 consensus threshold, 96% of LoveFraud conversations have at least one Group 3 trigger fire within the first 50 messages. The 3/3 unanimity threshold preserves coverage but pushes the median first-flag message to position 10, consistent with the trigger-disagreement analysis: models converge on the FSM state boundary but split on whether the same message is best described as Credential_Positioning or as Rapport_SelfDisclosure.

Table 20: End-to-end conversation SCAM-F1 under 5-fold nested CV (mean $\pm$ std over the 15 fold-seed estimates). Bold marks each encoder’s pick.

Encoder	$k=0$	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	$k=10$	$k=15$
DistilBERT 512	.820	.862	.846	.846	.856	.857	.847	.836
ModernBERT-SDPA 512	.845	.860	.863	.859	.863	.865	.860	.849
ModernBERT-SDPA 8k	.840	.858	.828	.865	.863	.856	.884	.852

Table 21: Token-length distribution for Stage 1 inputs.

k	Median	p90	p99	% >512
0	17	40	92	0.01%
5	107	204	352	0.26%
15	253	483	816	7.77%

Table 22: Checkpoint size and parameter count for selected models.

Artifact	Size	Params
DistilBERT Stage 1	253.2 MiB	66.4M
ModernBERT-SDPA Stage 1	568.5 MiB	149.0M
GRU Stage 2	2.3 MiB	0.604M
LSTM Stage 2	3.1 MiB	0.805M

Table 23: Annotator-side detection latency on 83 LoveFraud conversations. “Earliest any” = first model to flag. “ $\geq 2/3$ ” = majority consensus.

Threshold	Coverage	Med. msg	Med. %
Earliest any model	83/83 (100%)	4	1.3%
$\geq 2/3$ consensus	82/83 (99%)	5	2.6%
3/3 unanimous	82/83 (99%)	10	6.0%